HBO-ICT: CSC Semester 3

&

Duaal: TICT-CL3AUT-23

Automation

Werkboek



Inhoudsopgave

1 Inleiding: 5
1.1 Theorie en tentamen 5
1.2 Voorbereiding 6
1.2.1 Introductie 6
2 Opdrachten 7
2.1 Scripting 1: Python en Sockets 7
2.1.1 Lesopdracht 7
2.1.2 Weekopdracht 8
2.1.3 Lesstof tentamen 10
2.1.4 Informatiebronnen 10
2.2 Scripting 2: Modules | Security 11
2.2.1 Om de les mee te beginnen 11
2.2.2 Lesopdracht 11
2.2.3 Weekopdracht 13
2.2.4 Lesstof tentamen 14
2.2.5 Informatiebronnen 14
2.3 Scripting 3: Database | Flask 15
2.3.1 Lesopdracht 15
2.3.2 Weekopdracht 15
2.3.3 Lesstof tentamen 18
2.3.4 Informatiebronnen 18
2.4 Scripting 4: Change Management via CI/CD 19
2.4.1 Introduction 19
2.4.2 Lab Description 19
2.4.3 Deliverables (summative) 27
2.4.4 Information 28
2.5 (VOLTIJD ONLY) Scripting 5: PowerShell + AI 29
2.5.1 Lesopdracht PowerShell 29
2.5.2 Lesopdracht AI Codex 30
2.5.3 Weekopdracht PowerShell (deel A) 31
2.5.4 Weekopdracht AI (Codex, deel B) 32
2.5.5 Lesstof tentamen 32
2.5.6 Informatiebronnen 33
2.6 (DUAAL ONLY) Scripting 5: PowerShell 34
2.6.1 Lesopdracht 34

2.6.2 Weekopdracht 34
2.6.3 Lesstof tentamen 36
2.6.4 Informatiebronnen 36
2.7 (DUAAL ONLY): Scripting 6: XML & AI 37
2.7.1 Lesopdracht XML 37
2.7.2 Weekopdracht XML 37
2.7.3 Lesopdracht AI met ChatGPT, Copilot en Codex 38
2.7.4 Lesopdracht AI Codex 39
2.7.5 Weekopdracht AI 40
2.7.6 Lesstof tentamen 42
2.7.7 Informatiebronnen 42
2.8 Tooling 1: 'Bare Metal' → OS Deployment 43
2.8.1 Lesopdracht 43
2.8.2 Weekopdracht 43
2.8.3 Lesstof tentamen 50
2.8.4 Informatiebronnen 50
2.9 Tooling 2: Configuration Management Using SaltStack 51
2.9.1 Lesopdracht 51
2.9.2 Weekopdracht 51
2.9.3 Lesstof tentamen 56
2.9.4 Informatiebronnen 56
2.10 Tooling 3: Configuration Management (NextCloud) 57
2.10.1 Lesopdracht 57
2.10.2 Weekopdracht 57
2.10.3 Lesstof tentamen 62
2.10.4 Informatiebronnen 62
2.11 Tooling 4: OS Virtualization using Containers 63
2.11.1 Lesopdracht 63
2.11.2 Weekopdracht 63
2.11.3 Lesstof tentamen 66
2.11.4 Informatiebronnen 66
2.12 Tooling 5: Test Automation 67
2.12.1 Lesopdracht 67
2.12.2 Weekopdracht 68
2.12.3 Lesstof tentamen 69
2.12.4 Informatiebronnen 70
2.13 Tooling 6: Power Automate 71

2.13.1 Lesopdracht 72
2.13.2 Weekopdracht 73
2.13.3 Lesstof tentamen 78
2.13.4 Informatiebronnen 78
2.14 Bijlage1: Antwoorden 2.1.1, opdracht 1 79
2.15 Bijlage2: Antwoorden 2.2.1, functies 81
2.16 Bijlage3: Install VMware Workstation 83

1 Inleiding:

“Automation” leer je pas echt goed kennen door ermee te werken op basis van “echte” praktijksituaties. Dat is ook wat we gaan doen deze cursus en dit werkboek helpt je erbij. Je vindt in dit document alle opdrachten die horen bij deze cursus. Elke les heeft ongeveer dezelfde opbouw:

- **Lesopdracht**: Dit is een opdracht die je uitvoert in de les, daardoor breng je de theorie direct in praktijk.
- **Weekopdracht**: Deze opdrachten mag je alvast thuis maken, maar dat hoeft niet: Je maakt deze opdrachten tijdens het college om de lesstof die de docent dan uitlegt gelijk te oefenen tijdens de les. Het is te veel werk om af te krijgen tijdens de les, dus thuis afmaken.
- **Lesstof tentamen**: Bestudeer deze informatie heel goed, want bij dit vak hoort een schriftelijk tentamen. De vragen kunnen uit dit werkboek, de presentaties en de documenten op Canvas komen. Kortom: Alles op Canvas is belangrijk hiervoor.
- **Informatiebronnen**: Dit bevat extra informatie over het onderwerp. Hierdoor begrijp je de stof beter.

We gaan ervan uit dat je gebruik kunt maken van een eigen laptop.

1.1 Theorie en tentamen

In de studentenhandleiding staat hoe deze cursus is opgebouwd.
Bij een aantal sprints hoort een theorietoets. Hiervoor moet je de documenten op Canvas bestuderen inclusief dit werkboek met het kopje “lesstof tentamen”). Als de sprint geen theorietoets kent kan het document in de volgende sprint worden getoetst.

Les	Onderwerp	Bestuderen
T1	Inleiding automation	1a- automation_isnt_just_for_scale.pdf (5 blz.)
T2	Configuration Management	2a- introduction-to-configuration-management.pdf (8 blz.)
T2	Declarative	2b- Declarative vs Imparative.pdf (4 blz.)
T3	Saltstack	3a- gettingstartedwithsaltstack.pdf (39 blz.)
T4	IaC	6a- Infrastructure as Code.pdf (3 blz.)
T5	IaC	6d- Infrastructure as Code OReilly.pdf (19 blz.)
T6	Containers	6b- Docker.pdf (12 blz.)




1.2 Voorbereiding

1.2.1 Introductie

Bij de labs maken we gebruik van PyCharm of Visual Studio Code (VSC), Python versie 3 (tenzij anders aangegeven), Rocky 9 (Linux), Windows Server 2022, Recente Windows 11 Client en recente VMWare Workstation op je eigen laptop.

Windows Server en Clients kan je hier downloaden:
https://portal.azure.com/#blade/Microsoft_Azure_Education/EducationMenuBlade
Kies bij clients bij voorkeur voor de “Education” versie, en bij servers voor de “Datacenter” versie.

Hoewel er verschillende mogelijkheden zijn geven wij de voorkeur aan de onderstaande configuratie:
Je eigen laptop met 8GB of meer geheugen, een interne SSD, meest recente “Vmware Workstation 17” voor Windows“, Python 3.x (meest recente) en PyCharm.
Wat betreft PyCharm: je kan PyCharm professional gebruiken. Als je dat wilt, lees dan hier hoe dat moet:
https://blog.jetbrains.com/blog/2018/09/18/free-jetbrains-licenses-as-part-of-github-student-developer-pack/
Wat betreft VMware Workstation, de licentie komt nu niet meer van de HU. Je kan de software zelf gratis downloaden. Weet je niet meer hoe, kijk dan naar: Bijlage3: Install VMware Workstation.

Heb je nog vragen? Stel ze gerust aan je docent...

Je bent zelf verantwoordelijk voor het creëren van de juiste omgeving, maar we helpen je wel een beetje op weg. Als je kiest voor een andere configuratie zoals Linux waar het kan, of het gebruik van Apple besturingssystemen, dan is dat niet verboden, maar je krijgt daarbij geen ondersteuning en je moet wel zorgen dat je de geboden opdrachten goed kan uitvoeren. Dat geldt ook voor het gebruik van een andere IDE zoals Visual Studio Code (VSC), misschien moet je iets meer zelf uitzoeken.

Let op: het boek is opgedeeld in “Scripting” en “Tooling”. Deze wisselen elkaar af, maar de volgorde van de lessen binnen scripting of tooling blijft gelijk.

2 Opdrachten

2.1 Scripting 1: Python en Sockets

System en Network Engineering: Je zou kunnen zeggen dat “als je iets echt goed wilt begrijpen dan moet je er code voor schrijven”. Immers, iemand die de code heeft gemaakt, heeft de stof begrepen. Daarom is het belangrijk voor een netwerk engineer dat je eens zelf een TCP/IP-connectie opzet en daar data overheen laat gaan, geen software installeren en gebruiken, maar zelf maken. Dat gaan we doen deze les.

2.1.1 Lesopdracht

Opdracht 1

Oef, Python programmeren is alweer een jaar geleden. Daarom eerst een paar oefening om het programmeren weer even “op te halen”:

1) Zet 5 namen in een lijst en ga er met een for-loop doorheen. Print alle namen onder elkaar.

2) Vraag een gebruiker om een getal. Als het getal > 100 dan print je “GROOT”, anders print je “klein” op het scherm.

3) Een auto is rood, de trein is geel en de ambulance ook trouwens. Zet deze informatie in een dictionary en print deze vervolgens op het scherm.

4) Gegeven de string: a=”The client connects to the server by calling socket()”. Print karakter 5 t/m 10 op het scherm.

5) Gegeven de string uit de vorige opdracht: print vanaf de laatste 8 karakters tot het einde.

6) Print alle letters uit de vorige opdracht met een spatie tussen elke letter (tip: gebruik for-loop en bij de printopdracht: end=” “)

7) Maak een tuple van 10 getallen en print deze gesorteerd van klein naar groot op het scherm. Print daaronder het totaal van alle getallen.

8) Maak een eindeloze loop. Vraag daarin de gebruiker om input. Als de gebruiker “stop” invoert, dan moet je uit de loop springen en is het programma afgelopen.

9) Maak een functie. De functie krijgt een getal mee en geeft het dubbele van dit getal terug als het wordt aangeroepen. Roep de functie aan voor de getallen 3, 60 en -3 en print het resultaat op het scherm.

10) Vraag de gebruiker om een getal. Probeer het getal 100 daarna door de opgegeven invoer te delen en print het resultaat. Als de invoer “0” was, dan crashed je programma. Zorg dat dit niet gebeurt (gebruik excepties).

11) Maak met notepad een tekstbestand van 3 regels. Open dit bestand in Python en print regel voor regel op het scherm. Zorg ervoor dat er een regelnummer staat voor elke uitvoerregel. Dus bv zo:
    1 blabladfalf dsfjsflksjf
    2 ddddf dsfjsklfjklsf
    3 sfdkfjdkfjdkfjdkfdkfjdkf

12) Maak onderstaande code werkend.

```
def doeIets(x):
    # als x kleiner dan 10, dan moet y zijn: "x is kleiner dan 10"
    # anders, als x groter of gelijk aan 10, maar kleiner dan 20: y="x tussen 10 en 20"
    # anders: y="x is groter dan 19"
    # geef de variabele y terug
# vraag de gebruiker om een getal
# maak er een integer van
# roep de functie aan. Wat de functie teruggeeft moet naar de variabele z
# druk z af op het scherm
```

13) Maak een lijst met daarin twee lijstjes met ieder 3 getallen. print uit de tweede lijst het tweede getal.

14) Print de huidige werkdirectory (linux: pwd). Tip gebruik de module os en functie getcwd…

Vind je dit lastig? Gebruik deze site: https://python.swaroopch.com/ of https://www.learnpython.org/ om op niveau te komen. Welke site je ook gebruikt: niet alleen lezen, maar vooral DOEN! Autorijden leer je ook niet alleen uit een boekje…

Opdracht 2

De leraar heeft je zojuist uitgelegd via de presentatie hoe je met sockets kan werken in Python. Probeer nu in tweetallen via Python een connectie te maken (TCP) en data te sturen vanaf de ene computer naar de andere.
Werk als volgt:
Lees eerste deze pagina https://www.geeksforgeeks.org/python/python-network-programming/
Gebruik de informatie uit de voorbeelden om het nu zelf te programmeren:

- Student 1 maakt de server (een while loop hoeft nog niet. Als je 1 zin kan ontvangen en printen op het scherm is het al goed).
- Student 2 maakt de client (maak een connectie en stuur een zin).

Opmerkingen hierbij:

- Gebruik je windows? Denk aan de firewall.
- Is je Server af? Test 'm eerst met telnet vanaf linux, ncat, op Putty (raw mode), of Putty vanaf de commandline: `.\putty -raw 127.0.0.1 -P 8888`
- Is het gelukt? Kijk dan of je als server ook nog informatie terug kan sturen naar de client of maak je eigen "whatsapp" – 1:1 chat client!

2.1.2 Weekopdracht

Functionele eis:

Je gaat een service bouwen in Python die luistert op een poort (TCP). Vanaf je management systeem kan je een verbinding maken met deze server en via de service op afstand software starten op de externe host. Op die manier kan je het systeem met deze Python-server op afstand "beheren". Natuurlijk moet je er voor zorgen dat deze service geen veiligheidsrisico's oplevert voor de host en daarom moet je de service beveiligen.

Je hebt nodig:

De code hieronder heb je nodig voordat je kan beginnen met deze opdracht. Het bevat een hele eenvoudige tcp-service waarmee de basis is gelegd voor deze opdracht. Je maakt dus alleen de server-code. De client code heb je niet nodig, want client ben je door bijvoorbeeld putty in raw mode te gebruiken.
Hieronder de code die je kan gebruiken om te starten met de opdracht. Code kopiëren vanuit een pdf gaat niet altijd goed, misschien staat op canvas ook een docx-bestand met de inhoud van dit werkboek, kopiëren gaat dan vaak (een stukje) beter:

```
import socket
import sys
import time
import subprocess

HOST = ''   # Alle beschikbare interfaces
PORT = 8888 # Willekeurige poort (denk aan firewall bij Windows Systemen)

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  # avoid TIME_WAIT error
s.bind((HOST, PORT))
s.listen(10)
print('> Socket luistert op poort:',PORT)

# Wacht op connecties (blocking)
conn, addr = s.accept()

# Er is een client verbonden met de server
print('> Verbonden met ' + addr[0] + ':' + str(addr[1]))

# De server meldt zich aan de client
conn.sendall(b'WelkomOpMijnServer, vertel me iets, dan zeg ik hetzelfde terug:\r\n')

# Wacht op input van de client en geef deze ook weer terug (echo service)
data = conn.recv(1024)
data=str(data.decode('ascii')).rstrip() # # Remove \r | \n | \r\n

print('> Client data ontvangen:'+data+'<eindeData>')
conn.sendall(b"jeStuurdeMijDezeData:"+data.encode())
conn.sendall(b'\r\n')
time.sleep(1) # nodig op sommige computers, voorkomt verbreken verbinding te vroeg

# Verbreek de verbinding en sluit de socket
conn.close()
s.close()
```

Stappenplan:
Bouw het script als volgt op en test steeds ieder onderdeel voordat je verder gaat!

1. Zorg allereerst dat je vanaf een andere client kan communiceren met de bovenstaande Python-service (mag in tweetallen). Let hierbij op de firewall en gebruik als client netcat, telnet (niet de windows telnet client, linux versie kan meestal wel) of putty (in raw mode). **Kortom, zorg dat dit script "werkt"**.

2. Als de service werkt: Er is geen foutafhandeling opgenomen in het script bij het maken, binden en luisteren op een poort. **Voeg goede foutafhandeling toe** aan de server voor deze stappen in het proces

en test ook dat dit werkt. (Je kan dit zelf testen door als PORT een getal op te geven dat al wordt gebruikt door windows, je krijgt een overzicht met “netstat -an”.)

3. In het script verbreekt de server de verbinding nadat de client iets heeft verzonden. Zorg ervoor dat de server blijft luisteren en echo-en tenzij de client het woord "stop" zendt, in dat geval mag het script stoppen. **Kortom: er is een loop nodig.**

4. Maak het nu mogelijk dat een client **een extern commando** verstuurt dat op de server wordt uitgevoerd. Dit is natuurlijk een veiligheidsrisico: als je niet oppast kan een ander je harde schijf formatteren! Implementeer dit daarom als volgt: Als de client het woord “CALC” verstuurt dan start je de calculatoren als de client het woord “NOTEPAD” verstuurt start je notepad. Start alle executables asynchroon.

5. Je hebt het aantal commando’s al beperkt, maar bedenk nog **een extra beveiligingslaag**, zodat niet iedereen deze commando's kan uitvoeren, waarmee de oplossing nog veiliger wordt. Bepaal zelf hoe je dit wilt doen.

6. Iedere keer als een programma wordt gestart moet dit worden gelogd in **een logfile op de server**. Vermeld telkens een regel met de datum/tijd en het commando dat werd gestart.

7. Optioneel: **Volg de communicatie met een packet analyzer** zoals whireshark (hoef je niet te laten zien aan je docent).

2.1.3 Lesstof tentamen

Wat moet je weten voor het tentamen over dit onderwerp, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie over dit onderwerp.
- **Lees** alle informatie die wordt aangeboden onder het kopje “Informatiebronnen”.
- **Zorg dat je een fout kan halen** uit de Python code waarin een eenvoudige TCP-server of TCP-client wordt getoond. Dat gaat je op het tentamen niet lukken als je je code vooral door AI laat maken!
- Zorg dat je zelf clients en servers hebt gemaakt zodat je **getoonde code van anderen goed begrijpt** en kan **aanpassen/verbeteren**.

2.1.4 Informatiebronnen

- Youtube: https://www.youtube.com/watch?v=sUzM-vIC-s4
- Perfecte website voor allerlei modules, dus dit ook https://pymotw.com/2/socket/tcp.html
- Alle details, simpel http://www.tutorialspoint.com/python/python_networking.htm
- simpele voorbeelden, wat uitgebreidere uitleg https://pythontips.com/2013/08/06/python-socket-network-programming/

2.2 Scripting 2: Modules | Security

2.2.1 Om de les mee te beginnen

Eerst nog even dit: Functies zijn belangrijk in ieder stuk Python code. Kortom, basiskennis ook voor CSC-ers. Hieronder daarom wat oefeningen van (relatief makkelijk) tot echt wel moeilijk!

1. Schrijf een Python functie die het hoogste getal teruggeeft van een lijst met getallen.
   Sample List : [8, 2, 3, -1, 7]
   Expected Output : 8

2. Schrijf een functie die een leeftijd als input heeft en die je geboortejaar teruggeeft (je mag een variabele year_now=2025 gebruiken), waarna je het print.
   Bijvoorbeeld, stel dat het nu 2023 is en de input voor de functie is: 50
   Expected Output: 1973

3. Zelfde als de vorige vraag, maar nu het huidige jaar berekenen (iets met import datetime, google maar even?).

4. Schrijf een Python functie om te controleren of een getal voorkomt in een bepaald interval. Gebruik als eerste regel van de functie deze regel:
   def check_number(number,start,stop):
   Expected Output: yes (of no)

5. Schrijf een Python functie om alleen de even getallen van een bepaalde lijst te printen.
   Sample List : [1, 2, 3, 4, 5, 6, 7, 8, 9]
   Expected Result : [2, 4, 6, 8]

6. Schrijf een Python functie die alle getallen in een lijst met elkaar vermenigvuldigd.
   Sample List : [8, 2, 3, -1, 7]
   Expected Output : -336

7. Schrijf een Python programma die een string omkeert.
   Sample String : "1234abcd"
   Expected Output : "dcba4321"

8. Schrijf een Python functie dat een getal krijgt als parameter en vervolgens controleert of dit getal een priemgetal is of niet.

Antwoorden achterin dit werkboek.

2.2.2 Lesopdracht

Je hebt in de les gezien hoe je een module kan maken met daarin je eigen functies.

1. Maak nu je eigen module “mymodule” en zorg ervoor dat je de functie “dobbelsteen” vanuit een python script kunt aanroepen.
2. Als dat werkt maak je nog een nieuwe functie in deze module met de naam “yesno”. Als je deze functie aanroept dan moet de functie altijd “yes” of “no” teruggeven. Welke van de twee je terugkrijgt vanuit de

functie is willekeurig. Tip: gebruik uit de module random de functie “choice” in je eigen code van de module (zie ook http://www.tutorialspoint.com/python/number_choice.htm).

3. Ten slotte: kies logische variabele namen en documenteer iedere functie zodat de gebruiker ook later nog snapt welke argumenten je kan meegeven en wat de functie teruggeeft.

4. Beheerders hebben vaak te maken met scripts waarin wachtwoorden staan. Niet zo veilig:
   - Maak een scriptje met de naam ‘`write_password.py`’. Dit script bevat de volgende functionaliteit: Vraag de gebruiker om zijn/haar wachtwoord. Schrijf dit wachtwoord in het bestand ‘`password.txt`’.

   - Maak nu een tweede scriptje met de naam ‘`read_password.py`’. Dit script bevat de volgende functionaliteit: Vraag de gebruiker om zijn/haar wachtwoord en print “`Wachtwoord OK`” als het wachtwoord juist is en “`Wachtwoord FOUT`” als het wachtwoord fout is. Je moet daarbij het bestand password.txt gebruiken.

   - Oei, het wachtwoord is leesbaar in het bestand, dat is natuurlijk niet zo veilig. Daarom moet je de beide scripts zodanig aanpassen dat niet het wachtwoord, maar een (veilige) hash van het wachtwoord wordt geschreven in het bestand ‘`password.txt`’.

5. Een hacker heeft een bestand ‘password.txt’ gevonden. Dit bestand bevat de volgende inhoud:
   `f00bdeb2cd9da240a57c951fdf1bcba509fd0cd83c5e5ad9a669de12`
   - De hacker weet dat de inhoud tot stand is gekomen met behulp van de onderstaande code waarbij op de plaats van de vraagteken een hoofdletter heeft gestaan. Weet jij welke hoofdletter dit zou is geweest?
     ```
     print(hashlib.sha224(b"?").hexdigest())
     ```

   - De vorige opdracht kon je nog wel oplossen door alle letters te proberen. Dat wordt een stuk lastiger bij de volgende opdracht. Het wachtwoord heeft nu namelijk drie letters. De onderstaande code vraagt je om 100x een oplossing te proberen.

     ```
     import hashlib
     p="326f13e25448c99ccae78b0afa1c84e6fae2d8a587976df3a86628ae"

     for i in range(100):
         ww=input("Welk wachtwoord van 3 letters hoort hierbij?")
         if hashlib.sha224(ww.encode()).hexdigest() == p:
             print("Je hebt het wachtwoord gevonden! Het was:",ww)
             break
         else:
             print("Helaas, nog een poging (poging",i,")")
     ```

     Pas nu de loop aan zodat je alle opties automatisch probeert dus van AAA tot ZZZ.
     Tip: Gebruik de onderstaande tabel. Daarin zie je dat A hetzelfde is als het getal 65. Je kan de A ook printen door chr(65) te printen. Zo kan je een loop maken van A-Z.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Null	32	20	Space	64	40	@	96	60	`
1	01	Start of heading	33	21	!	65	41	A	97	61	a
2	02	Start of text	34	22	"	66	42	B	98	62	b
3	03	End of text	35	23	#	67	43	C	99	63	c
4	04	End of transmit	36	24	$	68	44	D	100	64	d
5	05	Enquiry	37	25	%	69	45	E	101	65	e
6	06	Acknowledge	38	26	&	70	46	F	102	66	f
7	07	Audible bell	39	27	'	71	47	G	103	67	g
8	08	Backspace	40	28	(	72	48	H	104	68	h
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o
16	10	Data link escape	48	30	0	80	50	P	112	70	p
17	11	Device control 1	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	50	32	2	82	52	R	114	72	r
19	13	Device control 3	51	33	3	83	53	S	115	73	s
20	14	Device control 4	52	34	4	84	54	T	116	74	t
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	End trans. block	55	37	7	87	57	W	119	77	w
24	18	Cancel	56	38	8	88	58	X	120	78	x
25	19	End of medium	57	39	9	89	59	Y	121	79	y
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	60	3C	<	92	5C	\	124	7C	|
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	□

TABEL 1: BRON:
HTTP://WWW.CDRUMMOND.QC.CA/CEGEP/INFORMAT/PROFESSEURS/ALAIN/IMAGES/ASCII1.GIF

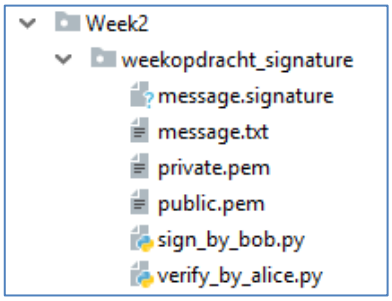
2.2.3 Weekopdracht

1. Gebruik je uitwerking van de lesopdracht om het nu allemaal netjes te maken. Wat zijn de eisen aan deze eerste opdracht uit de weekopdracht:
    - Maak de module “gamble” (file gamble.py).
    - Maak in deze module ten minste 5 functies en geef deze een logische naam:
        - Een dobbelsteen (1-6)
        - Een “kop” of “munt” optie, alsof je een munt in de lucht gooit
        - Een functie die een getal onder de 100 teruggeeft
        - <Eigen idee>
    - Documenteer elke functie zodat de gebruiker kan lezen wat de functie doet, wat er in gaat en wat de functie teruggeeft.
    - Maak een tweede file in python, bv test.py en vanuit die test.py roep je de vier verschillende functies aan zodat je kan testen over de module goed werkt.

2. Hier gaan we digitaal ondertekenen. Je wilt Alice een bestand sturen met de onderstaande inhoud:
   Hi Alice, mijn bankrekeningnummer is NL43RABO032233445, maak maar 870 euro over voor onze vakantie naar deze IBAN. Hartelijke groet, Bob
   Je wilt natuurlijk niet dat tijdens het transport van het bestand het bankrekeningnummer kan worden aangepast. Dit kan je voorkomen door het bestand te ondertekenen. Maak daarom 2 python scripts: Een script dat een keypair maakt en vervolgens het bestand leest en er een digitale ondertekening bij maakt.

Een script dat de digitale ondertekening leest en met behulp van de publieke sleutel de ondertekening controleert. Als de ondertekening in orde is, print je "`Signature OK`". Als het bestand is aangepast print je "`Signature NOK!!!`".

Een paar tips om je te helpen:
Gebruik deze bestandsnamen:



Hoe een key pair (public + private) moet maken vind je hier:
https://pycryptodome.readthedocs.io/en/latest/src/examples.html#generate-public-key-and-private-key
Je hebt de module "pycryptodome" nodig.

Hoe je met een key pair een bestand kan ondertekenen vind je hier:
https://pycryptodome.readthedocs.io/en/latest/src/signature/pkcs1_v1_5.html?highlight=signature
Let op, je moet de tekst eerst in een bestand zetten bv. met notepad in het bestand "message.txt", daarna lezen als binary file:

```
f = open(message_file, "rb")  # 'rb' !
```

en daarna ondertekenen.
Het resultaat", de variabele "signature" moet je naar een bestand schrijven. Doe dit weer binary, dus bijvoorbeeld: `f=open(signature_file,'wb')`

Hoe je een signature kan controleren lees je ook hier:
https://pycryptodome.readthedocs.io/en/latest/src/signature/pkcs1_v1_5.html?highlight=signature
Let op, lees nu ook weer eerst je message.txt bestand (binary mode weer), maak dan de hash en kijk of de hash gelijk is zoals beschreven op de webpagina.
Er staat wel een foutje op de webpagina bij de regel waar de hash wordt gemaakt. Dit moet zijn:

```
h = SHA256.new(message) # not SHA as written in the documentation
```

2.2.4 Lesstof tentamen

Wat moet je weten voor het tentamen over dit onderwerp, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie over dit onderwerp.
- **Leer** de algemene theorie zoals te lezen op: http://www.laurentluce.com/posts/python-and-cryptography-with-pycrypto/ (de python code hoef je niet te kennen, maar je moet de code wel kunnen begrijpen).

2.2.5 Informatiebronnen

Modules: https://docs.python.org/3/tutorial/modules.html
Security: http://www.blog.pythonlibrary.org/2016/05/18/python-3-an-intro-to-encryption/

2.3 Scripting 3: Database | Flask

Tot nu toe heb je gegevens opgeslagen in een bestand, in memory of in een CSV-bestand. Voor bepaalde toepassingen, zoals het schrijven van logging is dat voldoende, maar als je applicatie wat groter wordt of je wilt bepaalde gegevens aanpassen, dan moet je gaan werken met een database.
Tijdens deze week gaan we dit oefenen. Dit gaat het meest eenvoudig met SQLite, maar de gebruikte technologie is ook goed toepasbaar voor grotere databases zoals MySQL.
Bovendien gaan we aan de slag met Flask: Een manier om webapplicaties te bouwen met behulp van Python. Voor beheerders komt dit goed van pas, want niet iedereen heeft Python geïnstalleerd op zijn/haar computer, maar een webbrowser heeft iedereen op elk platform.
Alles bij elkaar kan je al een prima applicatie bouwen: website als front-end, python code voor de functionaliteit en de data wordt opgeslagen in een database!

Voorbereiding

- Installeer SQLite op je computer: https://www.sqlite.org/download.html . Kies in het geval van Windows niet voor de DLL, maar voor de zip-file waar sqlite3.exe in is te vinden.
- Daarnaast kan je nog een grafische manager voor de SQLite database installeren, zoals bijvoorbeeld de Firefox add-on https://addons.mozilla.org/nl/firefox/addon/sqlite-manager-webext

2.3.1 Lesopdracht

Misschien ook al genoemd in de presentatie:

- Installeer SQLite op je computer. Gebruik daarvoor de package manager “dnf” als je linux gebruikt.
- Maak een SQLite database met daarin een enkele tabel voor het opslaan van Naam/Adres/Woonplaats data. Gebruik hiervoor de SQLite3.exe console applicatie.
- Vul de tabel met drie regels (dus ook drie personen).
- Klaar? Maak nu een Python programma waarin je iemand vraag om zijn/haar voornaam. Daarmee zoek je de bijbehorende gegevens zoals adres en woonplaats op en toon deze informatie vervolgens op het scherm. Gebruik eventueel de informatie van: https://docs.python.org/3.8/library/sqlite3.html

2.3.2 Weekopdracht

SQLite

1. Maak een script dat random 1000 getallen genereert. De getallen moeten >0 en <500 zijn. Zorg ervoor dat deze getallen in een SQLite database worden geplaatst. Tabel heeft als definitie: CREATE TABLE nummers (getal INTEGER)
2. Zorg ervoor dat deze getallen *middels een database query* worden **opgeteld** en print het resultaat op de console.
3. Print daaronder het **gemiddelde** van deze getallen daarnaast.
4. Print daaronder het laagste en hoogste getal (ook dat kan met een database query).

Flask

Met de module Tkinter kan je een mooie GUI maken die platformonafhankelijk is. Helaas is dat niet zo heel eenvoudig en bovendien draait je applicatie alleen maar als Python is geïnstalleerd. Daarom is het soms verstandiger om je Python applicatie als webapplicatie aan te bieden, dan is het direct toegankelijk vanaf iedere client. Dit is precies wat we gaan doen tijdens deze opdracht.
Door de module Flask te importeren in je applicatie kan je een Web Applicatie maken zonder dat je je heel erg druk hoeft te maken over low level details zoals protocollen, thread management enzovoorts. Flask is een

zogenaamd “web applicatie framework”. Daar bestaan er meer van zoals Django en Bottle, maar Flask is eenvoudig om te leren en wordt vaak gebruikt. Je installeert het in Pycharm, of op de cli via:

```
pip install Flask
```

1. Probeer het onderstaande script te laten werken op Windows of Linux. Je ziet daar ook wat nieuws beginnend met het apestaartje: `@app.route('/')` Dit noemen we een decorator. Hierdoor kan je de code die eronder staat extra functionaliteit geven. Het vertelt de Flask webapplicatie in dit geval welke URL hoort bij welke functie die eronder staat. Dus ga je naar de root van de website (=’/’), dan wordt de functie `hello_world` aangeroepen. Probeer maar! Maak een python file, maar noem deze niet flask.py! (immers dan kan python de Flask module niet meer vinden).

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
   return 'Hello World'

if __name__ == '__main__':
   app.run()
```

Dus de “run()” methode heeft nog extra argumenten die je kan gebruiken: `app.run(host, port, debug, ssl_context)` . Even uitleggen:

- *host*: default is dit 127.0.0.1 Als je 0.0.0.0 gebruikt luister de webserver op al je ip adressen van de host
- *port*: default is dit 5000, maar je kan het aanpassen
- *debug*: default is dit False, kan ook True zijn natuurlijk om fouten te vinden.
- *ssl*_context: default is er geen ssl, maar met ‘adhoc’ wordt er een certificaat en key aangemaakt en heb je een https webserver ter beschikking. Je kan ook eigen sleutelmateriaal gebruiken trouwens. Werkt het gemakkelijkst onder linux.

Ok, we hebben een webserver. Je hebt nu ook content nodig die je kan serveren. Hieronder de vorbeeld html-code die je hiervoor nodig hebt. Kan jij deze html-pagina bereiken met je webbrower? Zorg wel dat deze in een “`templates`” -subdirectory van je python code komt te staan, anders kan je applicatie deze niet vinden.

De inhoud van de file `templates\index.html` is:

```
<HTML>
 <HEAD><TITLE>Python Sample Flask</TITLE></HEAD>
 <BODY>
  <H1>This is a header</H1>
  <p>See this is just like most other HTML<br>
 </BODY>
</HTML>
```

Pas je **python programma file** eerst aan zodat hij je html pagina ook zal gebruiken in de root van de website:

```
@app.route('/')
def index():
    return render_template('index.html')
```

2. Mooi, dit is dus een eenvoudig alternatief voor IIS of Apache, althans voor kleine beheerwebsites die niet door duizenden users gebruikt worden. Maar nu willen we ook gebruikers invoer verwerken: We hebben een webformulier nodig. Hoe zie dat er dan uit? Hieronder zie je alle informatie de je nodig hebt. Kan jij het werkend krijgen?

a. gebruiker-request.html

```
<html>
<head>Hallo Wie?</head>
<body>
<form action="../gebruiker.py" method="post">
    First Name: <input type="text" name="first_name"><br/>
    Last Name: <input type="text" name="last_name"/>
    <input type="submit" value="Submit"/>
</form>
</body>
</html>
```

b. gebruiker-response.html

```
<html>
<head>
    <title>Hello - Second Flask Program</title>
</head>
<body>
<h2>Hallo: {{ result['first_name'] }} {{ result['last_name'] }} !!!</h2>
</body>
</html>
```

c. gebruiker.py (De file is niet 100% klaar, met de kennis van hierboven kan je het werkend krijgen).

```
from flask import Flask, render_template, flash, request

@app.route('/')
def index():
    return render_template('gebruiker-request.html')

@app.route('/gebruiker.py', methods=['POST'])
def gebruiker():
    result = request.form
    return render_template('./gebruiker-response.html', result=result)
```

3. Nu komt de echte opdracht. Je moet nu zelf wat meer aan de slag. Maak je eigen -eenvoudige- webmail-applicatie. Dat wil zeggen: Maak een HTML-webformulier waarin je een ontvanger, subject en tekstveld voor de body kunt invoeren en zorg ervoor dat het bericht per mail kan wordt afgeleverd.

4. Zorg er met behulp van AI voor dat de applicatie nog wat meer functionaliteit krijgt. Denk bijvoorbeeld aan: goede lay out, attachment kunnen verzenden, nog wat meer opties zoals meerder ontvangers, een CC/BCC en een standaard handtekening mee sturen, kijkt maar wat mogelijk is.

   Wat betreft het verzenden van e-mail. Hieronder wat adviezen:
   - Stuur alleen mail naar jezelf.
   - Gebruik als afzender liever geen MS-adres omdat je daarvoor eerst een “Azure App Registration moet maken”. Makkelijker is het gebruik van een **Gmail-adres** (misschien een testaccount bij Google maken hiervoor?!).

- Om een e-mail naar een Gmail-adres te sturen met Python, kan je de “smtplib” importeren en poort 587 en STARTTLS gebruiken. Google heeft Basic Authentication afgeschaft, maar je kunt dit eenvoudig oplossen door een app-wachtwoord te gebruiken (als je tweestapsverificatie hebt ingeschakeld).
- Hoe doe je dat?!: eerst tweestapsverificatie inschakelen voor je account als je dat nog niet had (*https://myaccount.google.com/security*) en daarna een app-wachtwoord aanmaken op: *https://myaccount.google.com/apppasswords* . Vervang vervolgens in je python code je google-wachtwoord door het wachtwoord van 16 tekens dat je hebt gekregen speciaal voor je python script.

2.3.3 Lesstof tentamen

Wat moet je weten voor het tentamen over dit onderwerp, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie over dit onderwerp.
- **Lees** de informatie die wordt aangeboden onder het kopje “Informatiebronnen”.
- **Zorg dat je een fout kan halen** uit de Python code waarin een Flask-script / HTML-pagina wordt getoond.
- Zorg dat je zelf Flask-scripts hebt gemaakt zodat je **getoonde code van anderen goed begrijpt** en kan **aanpassen/verbeteren**.
- Begrijp hoe je veilig moet programmeren met databases om SQL-injectie te voorkomen.
- Zorg dat je **alle stappen kent** van het maken van een SQLite-database en -tabel tot de INSERT en SELECT van data.

2.3.4 Informatiebronnen

SQLite: https://docs.python.org/3.8/library/sqlite3.html
SQLite: https://www.youtube.com/watch?v=o-vsdfCBpsU en https://www.youtube.com/watch?v=qfGu0fBfNBs
Flask: https://www.tutorialspoint.com/flask/index.htm
Flask: https://www.digitalocean.com/community/tutorials/how-to-use-web-forms-in-a-flask-application
Mail sturen vanuit Python met een outlook account: https://pythonassets.com/posts/send-email-via-outlook-and-smtp/

2.4 Scripting 4: Change Management via CI/CD

This lesson's goal is to create a simple "continuous delivery pipeline" and be able to make this magic happen (sorry: English only 😟).

2.4.1 Introduction

Create your own CI/CD pipeline. Steps:

1. Write a little Python program (not Hello World)
2. Add some automated tests for the program
3. Push your code to GitHub
4. Activate GitHub Actions to check your code quality and run tests
5. Turn the Python program into a web app
6. Deploy the app to MS Azure
7. Make Sure that it continuously is integrated, deployed and delivered after each code change!

2.4.2 Lab Description

- **Preconditions**
  - Python3 on your laptop and a text editor like notepad. This tutorial assumes you're working in a Linux environment, e.g. WSL2, but Windows should be working as well.
  - Account on Github.
  - Access to MS Azure: https://portal.azure.com If you do not have access to Azure, create another CI/CD solution. Your options:
    1. Use Jetbrains Studentpack https://blog.heroku.com/github-student-developer-program)
    2. Use AWS Beanstalk (for access to AWS: send an email to rienk.vanderploeg@hu.nl requesting access to Learner Lab-Associate Services), tutorial: https://medium.com/swlh/get-started-with-ci-cd-using-github-actions-ca32d34b2943
    3. Create your own ci/cd solution using source control, web platform and python.

- **First step: Create an app directory structure.**
  Create directories and (empty) files:

```
cicd-buzz/
  requirements.txt
  app.py
  .gitignore
  tests/
    test_generator.py
  buzz/
    __init__.py
    generator.py
```

- **Next step: fill the file generator.py with python content, this is our complete application:**

```
from __future__ import print_function
import random

buzz = ('continuous testing', 'continuous integration',
    'continuous deployment', 'continuous improvement', 'devops')
adjectives = ('complete', 'modern', 'self-service', 'integrated', 'end-to-end')
adverbs = ('remarkably', 'enormously', 'substantially', 'significantly',
    'seriously')
verbs = ('accelerates', 'improves', 'enhances', 'revamps', 'boosts')

def sample(l, n = 1):
    result = random.sample(l, n)
    if n == 1:
        return result[0]
    return result

def generate_buzz():
    buzz_terms = sample(buzz, 2)
    phrase = ' '.join([sample(adjectives), buzz_terms[0], sample(adverbs),
        sample(verbs), buzz_terms[1]])
    return phrase.title()

if __name__ == "__main__":
    print(generate_buzz())
```

Test your app by executing the file a couple of times: **`python3 buzz/generator.py`**

- **Next step: Add automated tests:**
Continuous delivery pipelines only makes sense when you have a significant amount of automated tests that prevents you from continuously shipping broken software. To get a proper set of unit-tests for our buzz generator fill the file 'test_generator.py' in the tests-directory:

```
import unittest

from buzz import generator

def test_sample_single_word():
    l = ('foo', 'bar', 'foobar')
    word = generator.sample(l)
    assert word in l

def test_sample_multiple_words():
    l = ('foo', 'bar', 'foobar')
    words = generator.sample(l, 2)
    assert len(words) == 2
    assert words[0] in l
    assert words[1] in l
    assert words[0] is not words[1]

def test_generate_buzz_of_at_least_five_words():
    phrase = generator.generate_buzz()
    assert len(phrase.split()) >= 5
```

To run the tests we'll be using the 'pytest' framework. To install pytest we'll be using a Python Virtual Environment ('virtualenv'). Don't worry, that's easier done than said. Run de "venv" python module and create the venv directories and files:

```
python -m venv /path/to/your/cicd-buzz/venv
```

This should create a new directory venv. To start using this environment type:

```
[cicd-buzz] $ source venv/bin/activate
(venv) [cicd-buzz] $
```

Next, create fill the file called 'requirements.txt' that lists the all dependencies including pytest for testing and Flask for making this a webapp:

```
click
Flask==3.0.3
itsdangerous==2.1.2
Jinja2>=3.1.2
MarkupSafe
py
pytest
Werkzeug>=3.0.0
flake8
```

Download all dependencies by executing pip:

```
(venv) [cicd-buzz] $ pip install -r requirements.txt
```

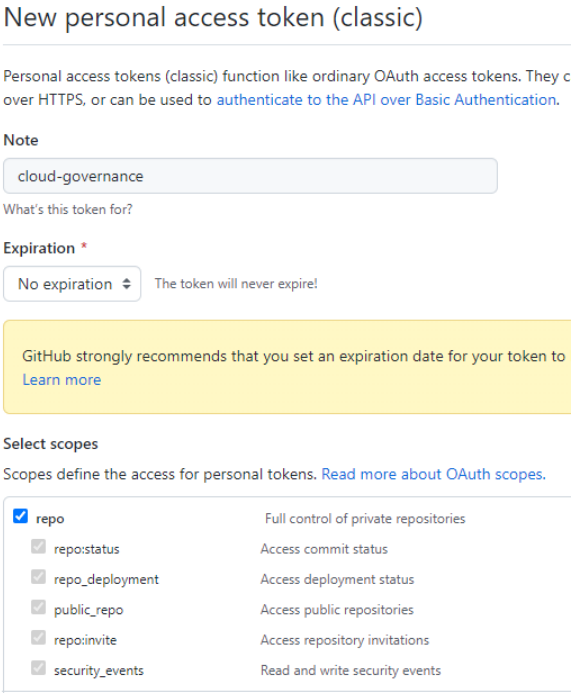
Inside the virtual environment you can now run the unit-tests in the 'test_generator.py' file:

```
(venv) [cicd-buzz] $ python -m pytest -v tests/test_generator.py
```

All three tests passed?

- **Next step: Push the code to GitHub**
  Login to GitHub (get an account first if you don't have one already) and create a new private repository called 'cicd-buzz'.

  Create a personal access token (classic) so that you can push your code to github using git:
  https://github.com/settings/tokens/new



Add **both** the "repo" and "workflow" scopes.
Don't forget to copy the token before you leave this screen!

Now it's time to initialize Git locally and push your code to GitHub. Firstly, put two lines in the file:
`.gitignore` preventing git to copy the whole virtual environment and compiled files to git:

```
venv
*.pyc
```

```
(venv) [cicd-buzz] $ git init    ;# sudo dnf install git?
(venv) [cicd-buzz] $ git add *
(venv) [cicd-buzz] $ git commit -m "Initial commit"
(venv) [cicd-buzz] $ git remote add origin
 https://<your-git-username>:<your-git-token>@github.com/<your-git-username>/cicd-
buzz.git
(venv) [cicd-buzz] $ git branch -M main
(venv) [cicd-buzz] $ git push -u origin main
```

Tip, use "**git status**" if your "lost"

- **Next: Make it a webapp**
  Nice Job! Now let's make this a web app. Since we will deploy the software to Azure as a web app, we first need to write a little Python Flask wrapper around our buzz generator to make the program respond to HTTP requests and output HTML. Add the code below in the file '**app.py**' in the root of your project directory:

```
import os
from flask import Flask
from buzz import generator

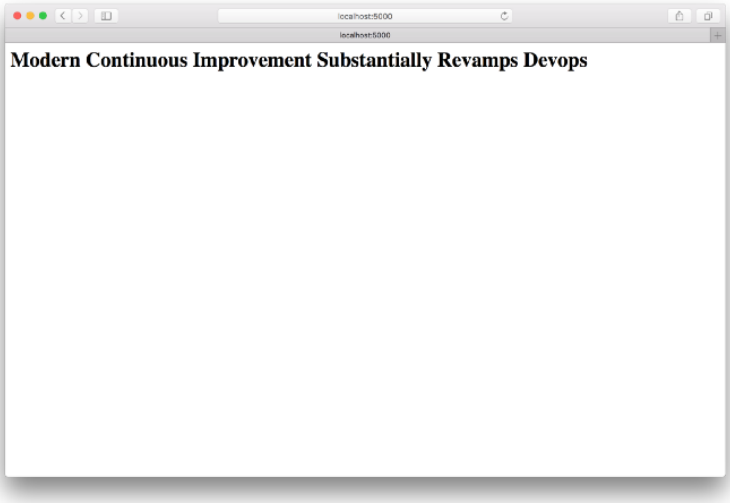
app = Flask(__name__)

@app.route("/")
def generate_buzz():
    page = '<html><body><h1>'
    page += generator.generate_buzz()
    page += '</h1></body></html>'
    return page

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=int(os.getenv('PORT', 5000)))
```

  You can now run the web app on your laptop. Access this from your browser and admire your achievement. Hit refresh a couple of times:

```
[cicd-buzz] $ python app.py
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
```



  Finally, don't forget to add your commit and push your changes:

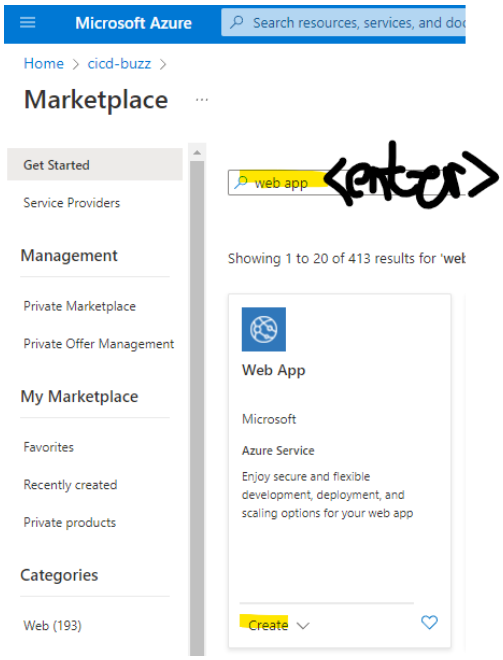
```
[cicd-buzz] $ git add app.py
[cicd-buzz] $ git commit -m "created web app"
[cicd-buzz] $ git push
```

  Does the contents of your repository on https://github matches your content on the disk?!

- **Next 6: Deploy the app to Microsoft Azure**
  Now we want to see it running on the Internet instead of working on localhost:5000 . This is possible using MS Azure. Create all resources in “West-Europe”.

  On Azure create a new “**resource group**”: “cicd-buzz”

  Within this resource group, create a new “**Web App**”, OS=Linux (=default), but change the Pricing Plan to **B1** if needed, this is costs about 13 euro / per month if you do not remove it after the lab. By the way, stopping the web app is not enough to stop paying money!



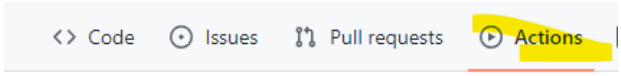
Some hints for Basic Tab:
  Publish: Code,
  Runtime: Python 3.14
  OS: Linux,
  Region: West Europe,
  Pricing Plan --> Basic B1

Some hints for Deployment Tab:
  Enable continuous deployment
  Connect Azure to your github account and select the right organization, repository and branch.

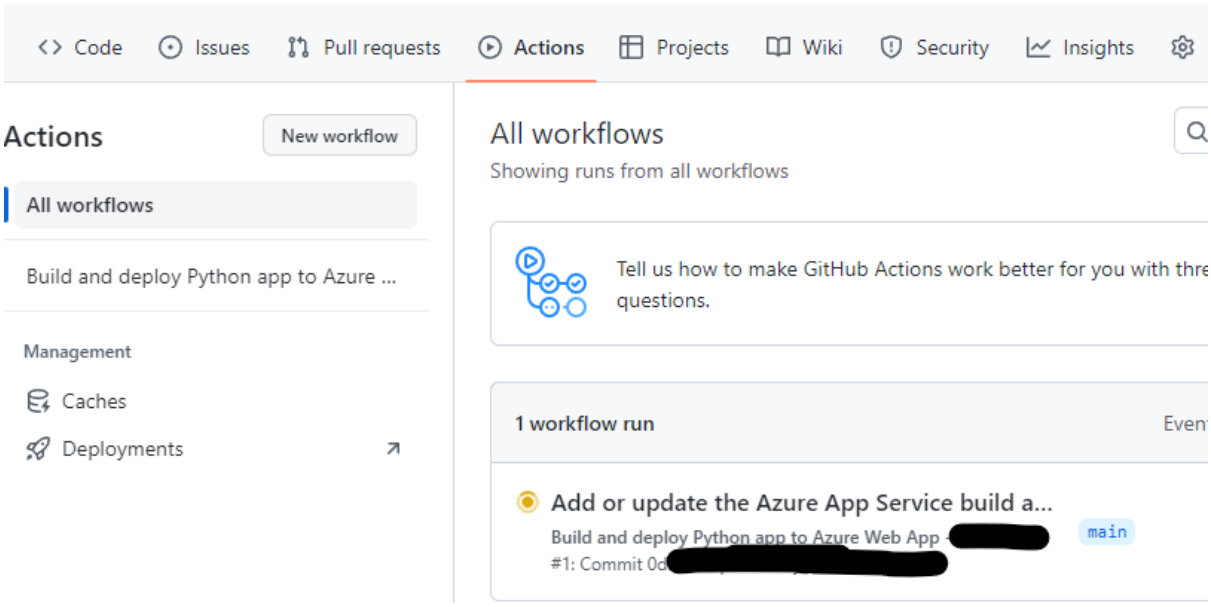
Create the deployment:

After a few minutes Azure adds a workflow file to the main branche with all jobs and steps that will be made after each code change. Take a look at: `.github/workflows/main_<appname>.yml`

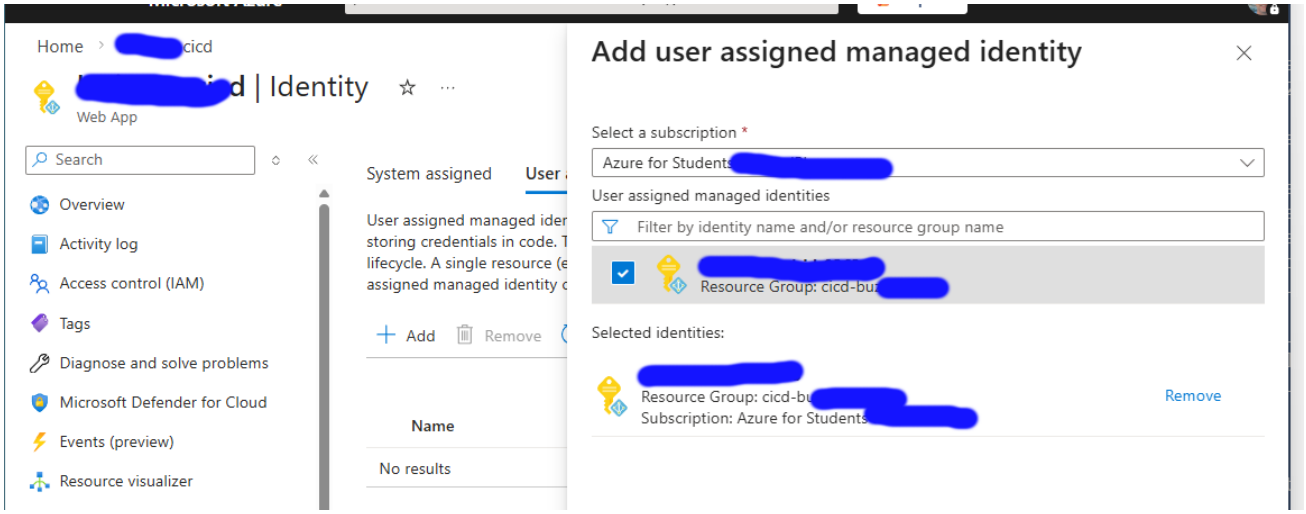
Also, take a look at the github actions:



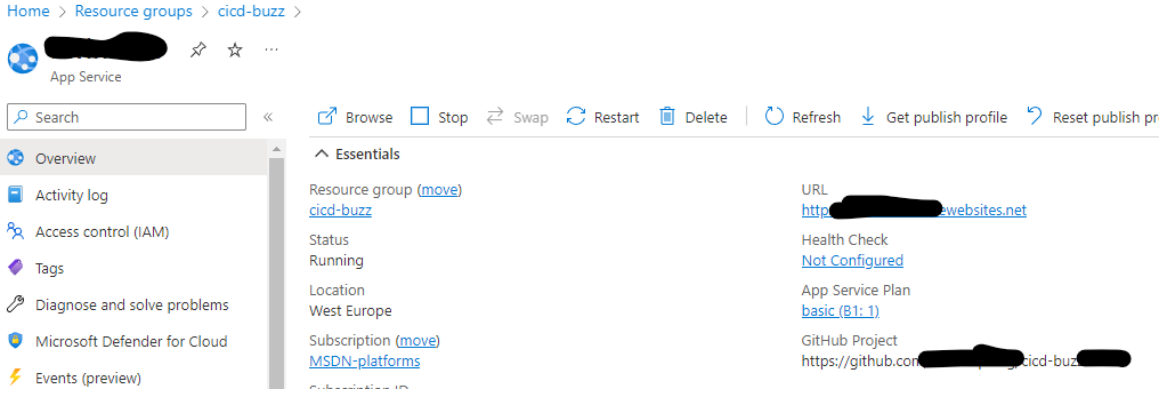
Click on “Actions” and on the workflow while running. Running takes a few minutes!



Errors while deploying?! (probably yes):
Create a "**user assigned" managed identity** making it possible voor Github Actions to deploy on Azure.



Visit the website after the workflow finished with a green check mark (URL is to be found as property of the App Service). **Spinning up the website also takes a few (up to ten) minutes**!

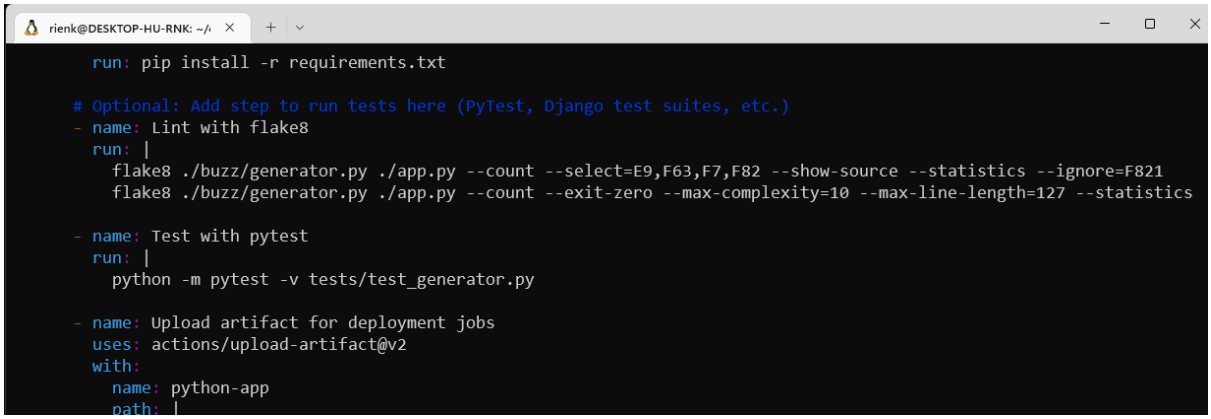


- **Step: Run tests and code quality checks automatically**
  Lets use Github Actions to run your tests and check your code quality on every commit.

  A few hints:
  - Issues with “git push”? Try “git pull” first to pull changes in the repository like the workflow file to your local environment.
  - You can find the github action file in the cicd-buzz/.github/workflows/ directory. The file starts with a dot (.), so its hidden, use ls -l to see the file.
  - Add two new “name”-sections the workflow file (e.g. .github/workflows/master_xxxxxx.yml) in the “build” job, after installing dependencies (requirements.txt) and before “upload artifacts” job.
  - Note, it is yaml, so beware of the number of spaces, see also screenshot below!:

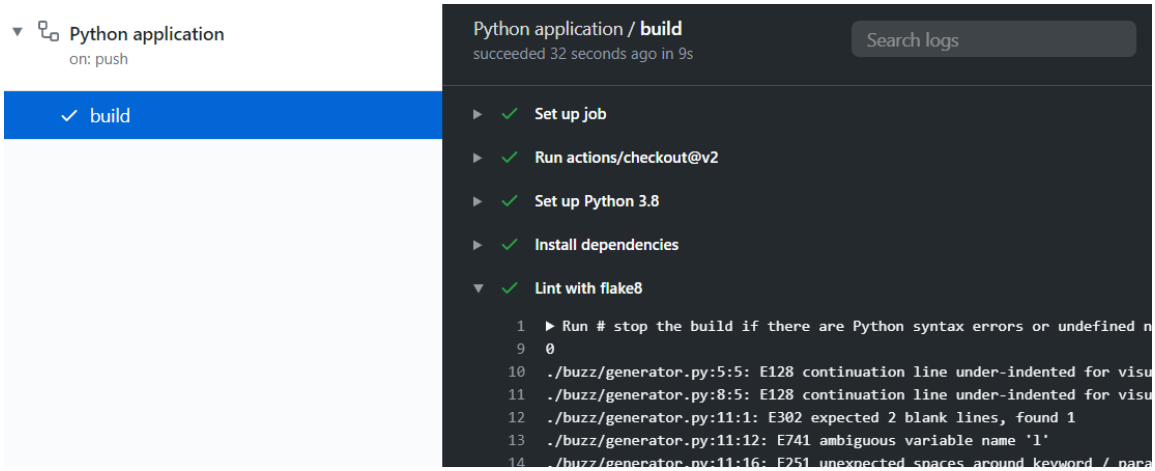
```
- name: Lint with flake8
  run: |
    source antenv/bin/activate
    flake8 ./buzz/generator.py ./app.py --count --select=E9,F63,F7,F82 --show-source --statistics --ignore=F821
    flake8 ./buzz/generator.py ./app.py --count --exit-zero --max-complexity=10 --max-line-length=127 -statistics

- name: Test with pytest
  run: |
    source antenv/bin/activate
    python -m pytest -v tests/test_generator.py
```



  Add the file, commit the change and push the code.

- **Step: Improve your code.**
  If you take a look at Github Action Workflow steps, your notice that “Lint with flake8” suggests some improvements.



Improve your code if you like, check it into git to see whether errors have been found. Note, after every code change, don't forget: `git add *; git commit -m "your message"; git push`

- **Step (optional): Add other actions, e.g. security checks.**
  You could use "bandit" (to search for known insecure Python constructions) and / or "pip-audit" to search for python dependencies that contain vulnerabilities.

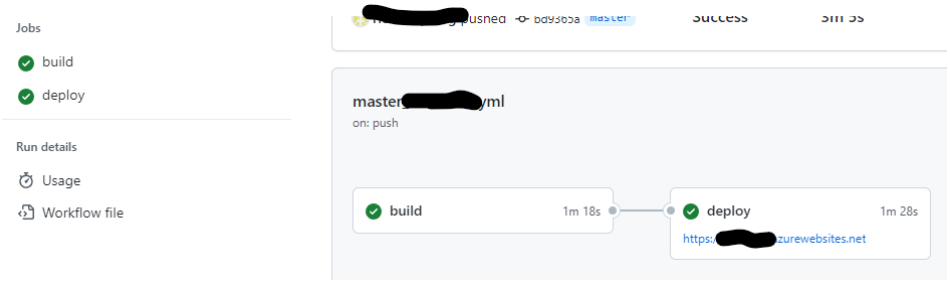
- **Final step: Continuous Integration is working fine?**
  Final: Make Sure that it continuously is integrated, deployed and delivered after each code change, e.g. by adding an extra line (replace it with your name) in app.py , you know where to put it I hope ?!

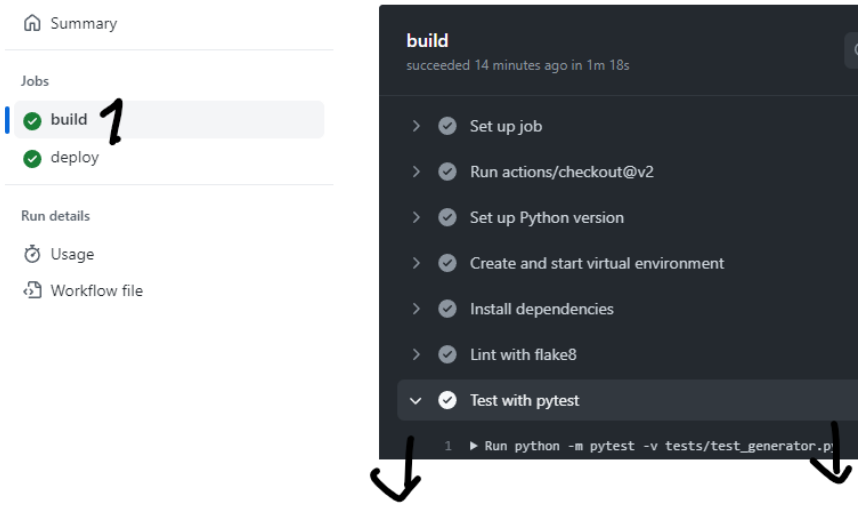
  ```
  page += "</br>Greetings from YourName </br>"
  ```

  And of course: Commit the change: (`git add *; git commit -m "print statement added"; git push`)

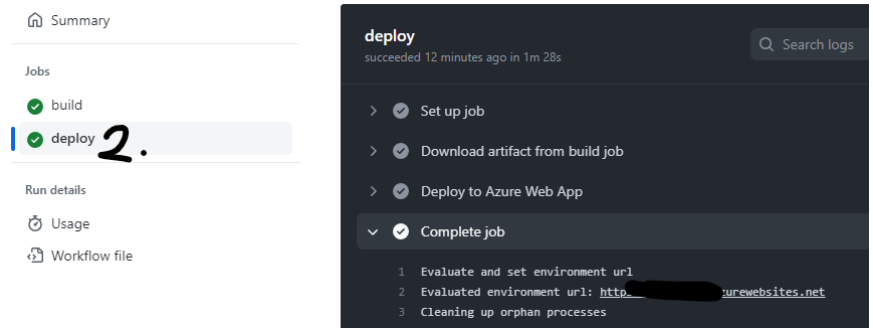
2.4.3 Deliverables (summative)

- **To prove that you successfully completed this lab** collect evidence, that is:
  1. Screenshot of output from: https://github.com/<username>/cicd-buzz/
  2. Screenshot of your webapp on Azure after you've done the final step.
  3. Screenshot containing the build phase using Github Actions





4. What is the complete output of the “Lint with flake8” phase?
5. What is the complete output of the “Test with pytest” phase? Do you see three times the word “PASSED”?
6. click on “deploy” and prove that there are no errors.



7.

- **Final:**
  Save your Azure credits! Delete the app service plan after all evidence is collected. To be sure: check the task on Canvas “Opdrachten”.

2.4.4 Information

- CI/CD Github: https://medium.com/bettercode/how-to-build-a-modern-ci-cd-pipeline-5faa01891a5b
- Github Actions: https://medium.com/swlh/automate-python-testing-with-github-actions-7926b5d8a865
- Heroku: https://stackabuse.com/deploying-a-flask-application-to-heroku/
- Beanstalk: https://medium.com/swlh/get-started-with-ci-cd-using-github-actions-ca32d34b2943
- Azure: https://github.com/MicrosoftDocs/azure-devops-docs/blob/main/docs/pipelines/ecosystems/python-webapp.md or https://swaydevstan.medium.com/deploy-a-python-web-app-using-azure-devops-d556a129164e

2.5 (VOLTIJD ONLY) Scripting 5: PowerShell + AI

PowerShell en AI: gekke combinatie. Idee is: PowerShell vooral in de les, AI op de HU voorbereiden en vooral thuis doen.

PowerShell is de manier om op het Windows platform administratieve taken uit te voeren. Meestal gebruik je hiervoor een “oneliner”, maar het is ook mogelijk om grotere scripts uit te voeren.

2.5.1 Lesopdracht PowerShell

Tijdens de les heb je al kennis gemaakt met PowerShell. Als je wat hebt geoefend, geef dan antwoord op de volgende vragen:

PowerShell Oneliners…

- Noem 4 Linux/Unix commando's die ook werken onder Powershell.
- Welk commando kan je ook gebruiken in plaats van "`cd`"?
- Welk commando kan je ook gebruiken in plaats van "`dir`"?
- “`cls`” is een alias van?
- maak een alias aan met de naam "`hier`" dat verwijst naar "`get-location`"
- "werkt" de alias nog als je het window met PowerShell afsluit en weer opnieuw opstart?
- Geef het commando waarmee je voorbeelden kan opvragen van het gebruik van "`stop-process`"
- Met welk commando kan je alle help updaten tot de meest recente versie?
- Met welk commando krijg je online hulp bij het commando "`stop-computer`" ?
- Wat gebeurt er als je een verplichte optie niet opgeeft? Bijvoorbeeld bij "`stop-process`" ?
- Laat alle processen zien met `ps` en geef deze weer in 4 kolommen.
- Laat alle processen zien met `ps` en geef deze weer als een lijst (label-waarde).
- Gebruik `test-connection` om een computer 4x te pingen met steeds 4 seconden tussenpauze.
- Een lastige: maak een lijstje aan met drie computer namen en sla deze op als "`hosts.txt`". Gebruik `test-connection` EN `get-content` om deze hosts achtereenvolgens te "pingen".
- gebruik de pipeline om de nieuwste 10 errors in de eventlog van het systeem te bekijken.
- Gebruik "`ps`" en converteer de output naar html
- Gebruik "`ps`" en converteer de output naar csv en sla deze op als file.
- Gebruik "`ps`" en selecteer verschillende kolommen waaronder `TotalProcessorTime`
- Gebruik "ps" en sorteer op `TotalProcessorTime` (aflopend). Welk proces gebruikt de meeste processortijd?
- Idem nu voor het gebruik van virtueel geheugen.
- Vraag een lijst met services op en groepeer op status. Hoeveel services zijn "running" en hoeveel "stopped"?
- Gebruik `measure-object` om het aantal entries in de `c:\windows` directory op te vragen.
- Start "`notepad`" een paar keer. Gebruik nu `ps` in combinatie met "`where`" en "`like`" om alle processen op te vragen die beginnen met een "n".
- Start `notepad`. Gebruik `get-process` en `stop-process` in een enkele pipeline om notepad te stoppen.

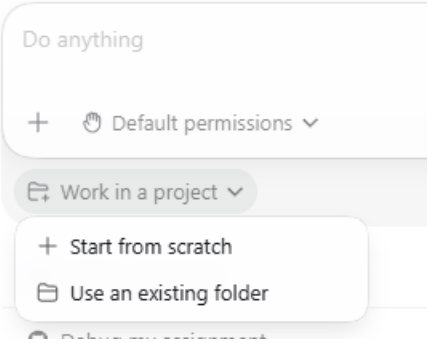
2.5.2 Lesopdracht AI Codex

Codex is de coding agent van OpenAI. Codex is niet alleen een model. Het is een product en ondersteunt workflow om je te helpen om code te schrijven, te reviewen en te debuggen. Of zoals OpenAI het zelf zegt, een AI coding agent die samen met je werkt om lokaal of in de cloud je (programmeer)taken uit te voeren.

Het is wat anders dan AI zoals in ChatGPT waar je meestal stukjes code krijgt als antwoord op je vraag. Codex kan in repositories kijken, files editen, commando's en tests uitvoeren. Je leert Codex het beste door het gewoon "te doen". Dat kan op de cli, dat kan als IDE extensie in VS Code, maar wij gebruiken het eenvoudig: de Codex App. Daar gaan we:

Opdracht 1: Installeer de Codex App

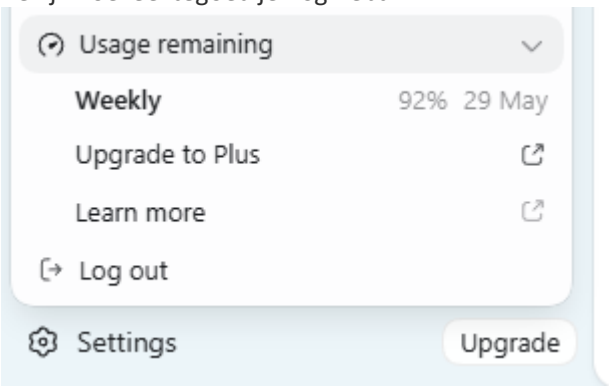
- Ga naar https://chatgpt.com/nl-NL/codex/ (voor macOS en Windows beschikbaar), dus niet via ChatGPT.
- Kies voor "Download voor Windows" en start de installatie vanuit de Windows Store (gebruik je die tenminste ook eens 😊). Later start je het programma opnieuw door "ChatGPT" te starten vanaf het start menu.
- Zorg dat codex staat integesteld voor "coding" --> settings --> general --> Work mode "For coding"
- Zorg op dezelfde plek dat de default permissies aan staan: Dus alleen read/edit files in de workspace.
- Om zaken eenvoudig te houden, maak een map op je computer genaamd "codex". In die map, maak een aparte map voor elk project. Als je wilt dat codex werkt met bestanden, sleep ze in de folder.



Opdracht 2: De eerste opdrachten

Begin een nieuwe chat en voer achtereenvolgens deze commando's uit. Wacht tussen door steeds op het resultaat en test het resultaat, werkt 't allemaal?

1. Prompt: "*maak tic, tac toe in python*"
2. Prompt: "*in welke directory staat het bestand*?"
3. Start het spel nu vanaf de powershell command line: `python tic_tac_toe.py`
4. Prompt: "*leuk, maar ik wil liever tegen de computer spelen*"
5. Prompt: "*nice! nu een nieuw spel, liever iets uit de lucht schieten spelletje of zo*"
6. Prompt: "*wat betekenen die cijfers (+ en - in kleur groen en rood) tijdens het genereren*"
7. Bekijk hoeveel tegoed je nog hebt.



2.5.3 Weekopdracht PowerShell (deel A)

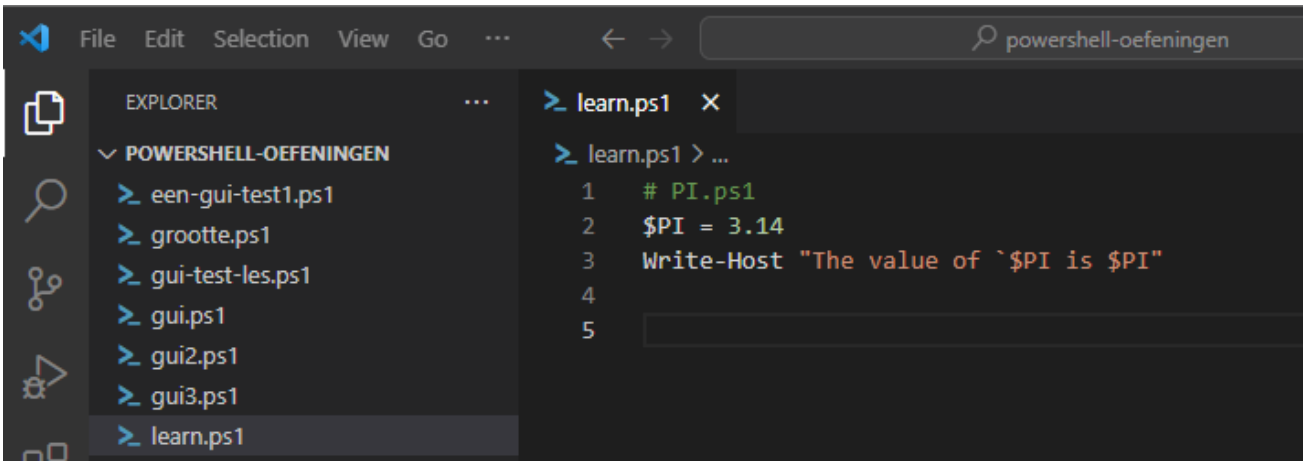
Zorg dat je eerste deze cursus hebt voltooid (behalve de laatste Azure module): https://learn.microsoft.com/en-us/training/paths/get-started-windows-powershell/

Doe de 3 modules. Die leveren je samen **2700** "ervaringspunten". Dit alles kost je zo'n 2 uur en 13 minuten volgens Microsoft. Je hoeft niet alles in de sandbox uit te voeren. Kan misschien ook wel lokaal.

Klaar? Maak een screenshot van de hele desktop waarin ik het browser scherm zie met daarin de ten minste 2700 XP punten.



Gebruik voor de oefeningen Visual Studio Code (VSC), of de PowerShell ISE. Kan allemaal lokaal.



Voorbeeld van PowerShell gebruikmakend van VSC.

PowerShell vanaf verschillende plaatsen

1. Maak een PowerShell script waarmee je de onderstaande functionaliteit realiseert:
    a. Vraag de gebruiker om een grootte (in MB) aan te geven.
    b. Zoek vanaf je "home directory" naar bestanden die groter zijn dan de grootte die de gebruiker aangaf.
    c. Sorteer deze bestanden op grootte en toon alleen de "top 10".
    d. Presenteer deze informatie op een overzichtelijke manier.
2. Run dit script vanuit **Visual Studio Code** (windows)
3. Run dit commando vanaf een linux command prompt (**pswh onder linux**)
4. Run dit commando vanaf de **cloudshell** op een vm vanaf portal.azure.com

Klaar? Maak **4 (1x script en 3x uitvoer) screenshots** en bijbehorende uitvoer van de powershell commando's als antwoord op de vragen hierboven.




2.5.4 Weekopdracht AI (Codex, deel B)

Ga nu zelf "lekker los". We maken een tool in python die je kan gebruiken als handig hulpmiddel voor allerlei CSC-taken, maar de eerste opdracht moet in PowerShell als bruggetje naar dit nieuwe onderwerp.

1: Ping Host (een GUI in Powershell)

Prompt: *"Maak een GUI geschreven in PowerShell waarmee ik een host kan pingen door de naam van die host op te geven."* Test de applicatie en als je verbeteringen of bugfixes wilt, vraag het aan de agent.

2: IP-bereik genereren (vanaf nu alles in Python)

Prompt: Maak een tool die "*die middels python een lijst van alle IP-adressen binnen mijn subnet 192.168.1.0/28 genereert."*

Test de tool vanaf de command line.

3: Port scanner.

Prompt: Maak een port scanner om uit te vinden of poort 22 (ssh) open staat voor een gegeven ip-adres.

4: LogAnalyse.

Wat gaan nuttige zaken halen uit logging. Voor beheerders is het niet te doen om alle logfiles uit te spitten. Het is beter om hier AI voor in te zetten! Moet je natuurlijk wel eerst een logfile hebben. "Gratis" logfiles kan je hier vinden: https://github.com/logpai/loghub Voor dit lab gebruiken we de logging van de SSH service: https://github.com/logpai/loghub/blob/master/OpenSSH/OpenSSH_2k.log

Vraag aan AI of deze code kan genereren voor een nieuwe tool, zodat je alle unieke gebruikers kan halen uit het logbestand. Nieuwe code genereren: vanaf welke ip adressen wordt geprobeerd in te loggen?

5: Complete beheertool: Nu zelf.

Zorg ervoor dat al de losse tools 2 (ip bereik) en 3 (port scanner) in één enkele tool komen met een menuutje en bedenk er zelf nog 2 functionaliteiten bij.

5: Lever nu het bewijs:

Screenshot 1: de ping host utility in PowerShell
Screenshot 2: het hoofdmenu van je beheertool
Screenshot 3: functionaliteit (met code zichtbaar in het screenshot) van één van de twee eigen functionaliteiten die je hebt toegevoegd (+uitvoer).

2.5.5 Lesstof tentamen

Wat moet je weten voor het tentamen over dit onderwerp, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie over dit onderwerp.
- Zorg dat je **de antwoorden weet op de PowerShell vragen** hierboven.
- Zorg dat je van PowerShell **oneliners begrijpt en kan uitleggen** wat er gebeurt.
- **Optioneel:** de informatie die wordt aangeboden onder het kopje "Informatiebronnen". Het gaat daarin niet om nieuwe informatie, maar om verbreding zodat je de stof beter begrijpt.




2.5.6 Informatiebronnen

- Microsoft Virtual Academy: https://mva.microsoft.com/en-US/training-courses/advanced-tools-scripting-with-powershell-3-0-jump-start-8277
- Algemene documentatie: https://learn.microsoft.com/en-us/powershell/
- Leuke oneliners: https://tseknet.com/blog/psoneliners

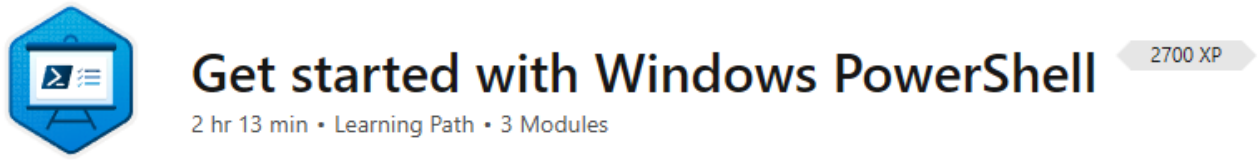
2.6 (DUAAL ONLY) Scripting 5: PowerShell

PowerShell is de manier om op het Windows platform administratieve taken uit te voeren vanaf de command line. Meestal gebruik je hiervoor een “oneliner”, maar het is ook mogelijk om grotere scripts uit te voeren.
In de opdrachten hieronder gaan we aan de slag met PowerShell.

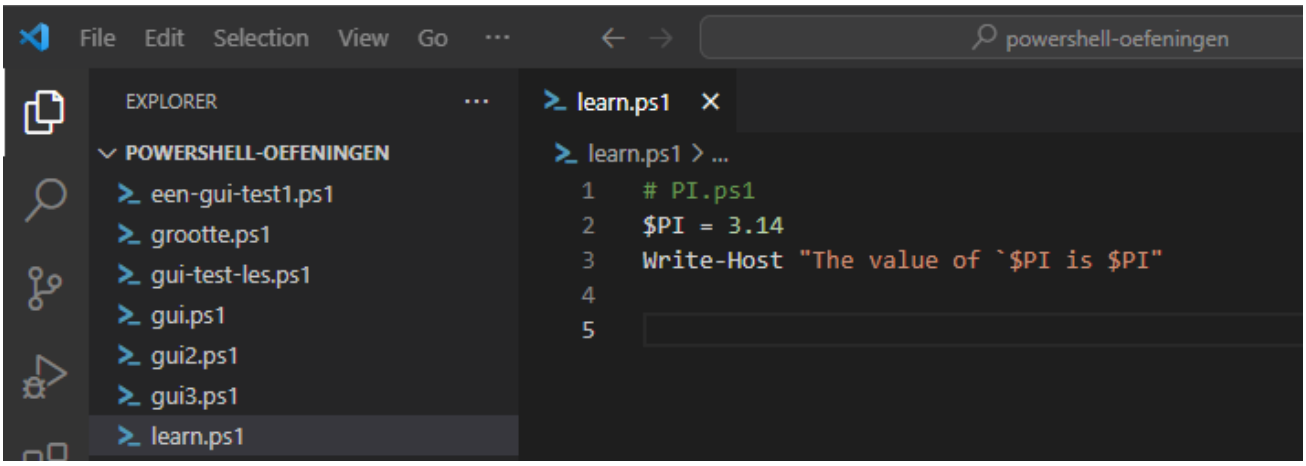
2.6.1 Lesopdracht

Tijdens de les heb je al kennis gemaakt met PowerShell. Ga nu zelf aan de slag door de online training van Microsoft te volgen. We gebruiken hiervoor https://learn.microsoft.com/ , het online platform van Microsoft om nieuwe technologie te leren. Volg de onderstaande cursus en zorg ervoor at je browser pop-ups toestaat van

De te volgen cursus is: https://learn.microsoft.com/en-us/training/paths/get-started-windows-powershell/
Doe de 3 modules. Die leveren je samen **2700** “ervaringspunten”. Dit alles kost je zo’n 2 uur en 13 minuten volgens Microsoft. Je hoeft niet alles in de sandbox uit te voeren. Kan misschien ook wel lokaal.
Klaar? Maak een screenshot van je HELE desktop scherm waarin ik ook kan zien dat je ten minste 2700 XP punten hebt behaald.



Zorg dat je powershell kan gebruiken vanuit Visual Studio Code (VSC), cd command lines (powershell core of desktop powershell command line onder windows of linux), PowerShell ISE en de Azure cloudshell.



Voorbeeld van PowerShell gebruikmakend van VSC.

2.6.2 Weekopdracht

Zorg dat je eerst de online cursus hebt voltooid.
Geef dan antwoord op de volgende vragen:

PowerShell Oneliners…

1. Noem 4 Linux/Unix commando's die ook werken onder Powershell.
2. Welk commando kan je ook gebruiken in plaats van "`cd`"?
3. Welk commando kan je ook gebruiken in plaats van "`dir`"?
4. "`cls`" is een alias van?
5. maak een alias aan met de naam "`hier`" dat verwijst naar "`get-location`"
6. "werkt" de alias nog als je het window met PowerShell afsluit en weer opnieuw opstart?
7. Geef het commando waarmee je voorbeelden kan opvragen van het gebruik van "`stop-process`"
8. Met welk commando kan je alle help updaten tot de meest recente versie?
9. Met welk commando krijg je online hulp bij het commando "`stop-computer`" ?
10. Wat gebeurt er als je een verplichte optie niet opgeeft? Bijvoorbeeld bij "`stop-process`" ?
11. Laat alle processen zien met `ps` en geef deze weer in 4 kolommen.
12. Laat alle processen zien met `ps` en geef deze weer als een lijst (label-waarde).
13. Gebruik `test-connection` om een computer 4x te pingen met steeds 4 seconden tussenpauze.
14. Een lastige: maak een lijstje aan met drie computer namen en sla deze op als "`hosts.txt`". Gebruik `test-connection` EN `get-content` om deze hosts achtereenvolgens te "pingen".
15. gebruik de pipeline om de nieuweste 10 errors in de eventlog van het systeem te bekijken.
16. Gebruik "`ps`" en converteer de output naar html
17. Gebruik "`ps`" en converteer de output naar csv en sla deze op als file.
18. Gebruik "`ps`" en selecteer verschillende kolommen waaronder `TotalProcessorTime`
19. Gebruik "ps" en sorteer op `TotalProcessorTime` (aflopend). Welk proces gebruikt de meeste processortijd?
20. Idem nu voor het gebruik van virtueel geheugen.
21. Vraag een lijst met services op en groepeer op status. Hoeveel services zijn "running" en hoeveel "stopped"?
22. Gebruik `measure-object` om het aantal entries in de `c:\windows` directory op te vragen.
23. Start "`notepad`" een paar keer. Gebruik nu `ps` in combinatie met "`where`" en "`like`" om alle processen op te vragen die beginnen met een "n".
24. Start `notepad`. Gebruik `get-process` en `stop-process` in een enkele pipeline om notepad te stoppen.

Klaar? Maak een screenshot van de powershell commando's als antwoord op de vragen hierboven.

PowerShell Script

1. Zorg ervoor dat je executionpolicy zodanig is aangepast dat je scripts kunt starten.
2. Maak dan een script met de naam "`np.ps1`" dat 3x notepad start, 3 seconden "slaapt", de proces-id's van notepad opvraagt en vervolgens alle notepads stopt.

Klaar? Maak een screenshot van het script als antwoord op de vraag hierboven. Test 't wel zelf eerst natuurlijk.

PowerShell vanaf verschillende plaatsen

5. Maak een PowerShell script waarmee je de onderstaande functionaliteit realiseert:
    a. Vraag de gebruiker om een grootte (in MB) aan te geven.
    b. Zoek vanaf je "home directory" (of een andere directory) naar bestanden die groter zijn dan de grootte die de gebruiker aangaf.
    c. Sorteer deze bestanden op grootte en toon alleen de "top 10".
    d. Presenteer deze informatie op een overzichtelijke manier.
6. Run dit script vanuit **Visual Studio Code** (windows)
7. Run dit commando vanaf een linux command prompt (**pswh onder linux**)
8. Run dit commando vanaf de **cloudshell** op een vm vanaf portal.azure.com

Klaar? Maak **4 (1x script en 3x uitvoer) screenshots** en bijbehorende uitvoer van de powershell commando's als antwoord op de vragen hierboven.

PowerShell in een windows domein

Het voordeel van PowerShell in het domein is dat authenticatie naar andere computers automatisch gaat.
Zorg voor een domein met daarin twee of meer windows computers. Je mag gebruik maken van een bestaand domein, of maak een nieuwe.

1. Log als administrator in vanaf de ene computer op de andere (gebruik `Enter-PSSession`) en vraag een proceslijst op van die remote computer.
2. Log weer uit en gebruik daarna "`Invoke-Command`" om de proceslijst van die tweede computer in één keer op te halen.
3. Probeer met powershell een proceslijst op te vragen van de 5 processen per computer die de meeste memory gebruiken en sorteer de totaallijst op memory gebruik. In de lijst moet ik het proces en het memory gebruik kunnen aflezen.

Klaar? Maak **een screenshot** van de powershell commando's en bijbehorende uitvoer als antwoord op de vragen hierboven.

2.6.3 Lesstof tentamen

Wat moet je weten voor het tentamen over dit onderwerp, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie over dit onderwerp.
- Zorg dat je **de antwoorden weet op de PowerShell vragen** hierboven.
- Zorg dat je van PowerShell **oneliners begrijpt en kan uitleggen** wat er gebeurt.
- **Optioneel:** de informatie die wordt aangeboden onder het kopje "Informatiebronnen". Het gaat daarin niet om nieuwe informatie, maar om verbreding zodat je de stof beter begrijpt.

2.6.4 Informatiebronnen

- Microsoft Virtual Academy: https://mva.microsoft.com/en-US/training-courses/advanced-tools-scripting-with-powershell-3-0-jump-start-8277
- Algemene documentatie: https://learn.microsoft.com/en-us/powershell/
- De training: https://learn.microsoft.com/nl-nl/training/paths/powershell/
- Leuke oneliners: https://tseknet.com/blog/psoneliners

2.7 (DUAAL ONLY): Scripting 6: XML & AI

XML en AI: gekke combinatie. Idee is: XML vooral in de les, AI op de HU voorbereiden en vooral thuis doen.

Tegenwoordig wordt op veel plaatsen YAML of JSON gebruikt, maar XML is overal: Dit document, opgeslagen als office-docx document, is een XML-bestand bijvoorbeeld. Beheerders moeten deze bestanden regelmatig lezen of aanpassen.
Tijdens het college gaan we aan de slag met XML. We leren dat het bestaat uit “gewoon” tekst. Toch is tekst helemaal niet zo “gewoon”. Als encoding wordt besproken snap je wat daar ingewikkeld aan is…

2.7.1 Lesopdracht XML

Een typische FTP-server gebruikt vaak een configuratiebestand in INI-formaat met secties, opties en waarden. Veel FTP-servers gebruiken echter hun eigen formaat (bijvoorbeeld vsftpd.conf of proftpd.conf). Onderstaande is een algemene voorbeeld-INI-configuratie voor een FTP-server, geschikt als oefenbestand. Herschrijf het hele bestand zodat het in XML is geschreven.

```
; ftpserver.ini
; Configuratiebestand voor een FTP-server

[server]
listen_port = 21
protocol = FTP

[authentication]
require_login = true
allow_guest = false

[general]
download_directory = /srv/ftp/downloads
log_file = /var/log/ftp/server.log
```

Neem de lesopdracht over de XML-configuratiefile voor de FTP-server. Schrijf nu in Python de juiste code, zodat de geconfigureerde informatie gelezen word en getoond wordt op het scherm.

2.7.2 Weekopdracht XML

1) Maak met een teksteditor als notepad/textpad/notepad++ een XML file. Doe dit door de onderstaande Cisco configuratiefile om te schrijven naar XML. Bekijk daarna het resultaat met je browser (liefst Chrome). Is je XML Well Formed? Maak logisch gebruik van elementen, nesting en attributen. Je mag de naamgeving van elementen/attributen aanpassen als je daar een goede reden voor hebt.

```
interface Ethernet0
 nameif test
ip address 10.10.88.50 255.255.255.254
!
interface Ethernet1
 nameif inside
ip address 10.86.194.176 255.255.254.0
!
enable password 8Ry2YjIyt7RRXU24 encrypted
passwd 2KFQnbNIdI.2KYOU encrypted
hostname XXX
!
```

```
policy-map xxx_global_fw_policy
 class inspection_default
  inspect dns
  inspect ftp
  inspect http
```

2) Voeg bij Interface Ethernet 0 een description element of attribuut toe met de onderstaande tekst en bekijk met de browser of dit is gelukt: `Herman's network: "<Games & Other stuff>"`

3) Voeg op een logische plaats commentaar toe aan de XML-file.

4) Voeg het onderstaande tekstblok ongewijzigd toe aan de XML-file door gebruik te maken van een CDATA element:

```
for (c in colors) {
      var item = $('<div/>').css("background-color",'#'+colors[c]).addClass("colorbox");
      item.click((function () {
        var col = colors[c];
        return function () {
          currentColor = col;
        };
      })());
```

5) Verander de description van interface Ethernet 1 in: 请收藏　Bekijk in Chrome of de tekst leesbaar is.

2.7.3 Lesopdracht AI met ChatGPT, Copilot en Codex

"AI vervangt geen programmeurs, maar programmeurs die AI gebruiken vervangen programmeurs die dat niet doen." In deze les gaan we bewust AI gebruiken zodat je de voordelen goed kan benutten, maar let op: In de theorietoets krijg je ook code te zien waarvan jij de werking moet uitleggen. Kortom, snap goed wat er staat en zorg dat je het zelf kan aanpassen.

Opdracht 1: Join GitHub Education

- Stap 1: Ga naar de pagina van het Student Developer Pack: https://education.github.com/pack
- Stap 2: Meld je aan voor het Student Developer Pack
- Stap 3: Selecteer 'Get Student Benefit'
- Stap 4: Meld je aan of registreer je voor een GitHub-account.
- Stap 5: Voeg het e-mailadres van je school toe. Dit proces vereist meestal authenticatie, omdat er een code naar het e-mailadres van je school wordt gestuurd ter verificatie.
- Stap 6: Voer de naam van je school in
- Stap 7: Beantwoord de vragen over hoe je GitHub wilt gebruiken.
- Stap 8: In sommige gevallen kan er meer bewijs nodig zijn. Je moet dan mogelijk een identiteitskaart van je school of een ander bewijs van je studieduur uploaden.
- Laatste stap: Dien de aanvraag in.

Ingediende aanvragen worden soms binnen een uur of maximaal een paar dagen beoordeeld.

Opdracht 2: Installeer Visual Studio Code

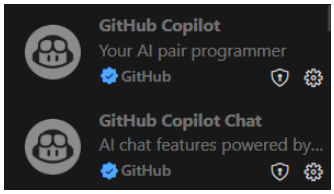
We gaan eerst de juiste omgeving installeren op je laptop. Om GitHub Copilot te gebruiken, heb je Visual Studio Code (VS Code) nodig:

1. Ga naar code.visualstudio.com.
2. Download en installeer de versie voor jouw besturingssysteem (Windows, Mac of Linux).
3. Open VS Code na de installatie.

Opdracht 3. Activeer GitHub Copilot

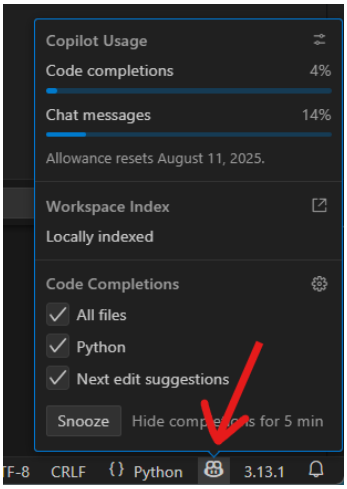
Zodra je Educatie-licentie is goedgekeurd:

1. Ga naar GitHub Copilot en klik op Start free trial
2. Koppel Copilot aan je GitHub-account.



Opdracht 4. Gebruik Copilot in Visual Studio Code

1. Open Visual Studio Code en ga naar de **Extensions Marketplace** (icoon links in de zijbalk) en zoek naar deze extensies: : **GitHub Copilot** - **GitHub Copilot Chat** .
2. Installeer de extensies en log in met je GitHub-account met de Educatie-licentie.
3. Na het inloggen kun je Copilot direct gebruiken: tijdens het typen krijg je slimme code-suggesties en kun je sneller voorbeelden, oefeningen en lesmateriaal maken.
4. Check of het goed is geïnstalleerd. Klik daarvoor op het icoon rechtsonder:



2.7.4 Lesopdracht AI Codex

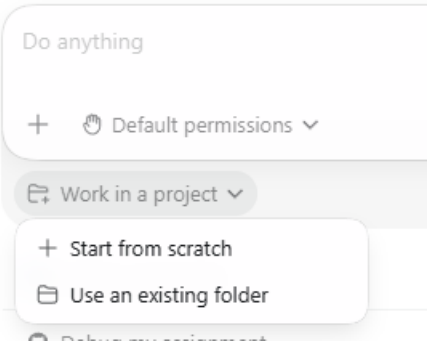
Codex is de coding agent van OpenAI. Codex is niet alleen een model. Het is een product en ondersteunt workflow om je te helpen om code te schrijven, te reviewen en te debuggen. Of zoals OpenAI het zelf zegt, een AI coding agent die samen met je werkt om lokaal of in de cloud je (programmeer)taken uit te voeren.

Het is wat anders dan AI zoals in ChatGPT waar je meestal stukjes code krijgt als antwoord op je vraag. Codex kan in repositories kijken, files editen, commando's en tests uitvoeren. Je leert Codex het beste door het gewoon "te doen". Dat kan op de cli, dat kan als IDE extensie in VS Code, maar wij gebruiken het eenvoudig: de Codex App. Daar gaan we:

Opdracht 1: Installeer de Codex App

- Ga naar https://chatgpt.com/nl-NL/codex/ (voor macOS en Windows beschikbaar), dus niet via ChatGPT.

- Kies voor “Download voor Windows” en start de installatie vanuit de Windows Store (gebruik je die tenminste ook eens 😊). Later start je het programma opnieuw door “ChatGPT” te starten vanaf het start menu.
- Zorg dat codex staat integesteld voor “coding” --> settings --> general --> Work mode “For coding”
- Zorg op dezelfde plek dat de default permissies aan staan: Dus alleen read/edit files in de workspace.
- Om zaken eenvoudig te houden, maak een map op je computer genaamd “codex”. In die map, maak een aparte map voor elk project. Als je wilt dat codex werkt met bestanden, sleep ze in de folder.



2.7.5 Weekopdracht AI

Zo, we zijn nu klaar voor de opdrachten in ChatGPT, Copilot en Codex, daar gaan we:

1: IP-bereik genereren. Gebruik ChatGPT

Vraag aan ChatGPT: *“Genereer code die een lijst van alle IP-adressen binnen mijn subnet 192.168.1.0/28 genereert.”*

Hmm, je moet wel python programmeur zijn om dit te kunnen lezen, we zijn nog niet helemaal tevreden: *“Genereer de uitvoer zo dat er een lijst onder elkaar komt met ip adressen. Bij elk ip-adres graag ook het subnet vermelden”.*

Accepteer het voorstel en voer de code uit. Werkt dit? Tevreden? Doe eventueel nog wat aanpassingen. **(bewaar het antwoord).**

2: Port scanner. Gebruik Copilot (met Visual Studio Code + Python extension + Copilot)

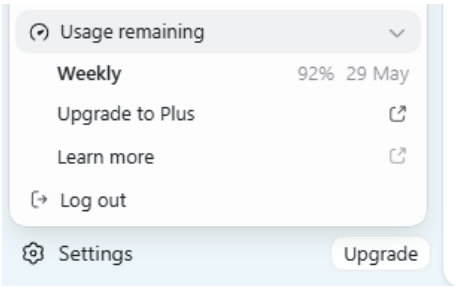
1. Maak een nieuwe python file aan en druk op `<ctrl-i>`. Type daar de volgende opdracht: *“Maak een poort scanner die controleert of poort ssh of rdp openstaat op een gegeven IP-adres”.*
   Accepteer het voorstel en voer de code uit. Plaats een hash (‘#’) op een logische plek in de code en zet erachter:
   `# scan all ips in subnet 192.168.1.0/28`
   `Druk dan op <enter> dan wordt de gewenste code gemaakt.`
2. Voer de code uit. Werkt dit goed?
   Vaak kan je met “tab” de code accepteren en met “esc” kan je de suggestie weer verwijderen.

Werkt dit? **(bewaar het antwoord).**

3: Tic, tac, toe. Gebruik Codex

Nu alles automatisch met Codex. Begin een nieuwe chat in Codex en voer achtereenvolgens deze commando’s uit. Wacht tussen door steeds op het resultaat en test het resultaat, werkt ’t allemaal?

1. Prompt: “*maak tic, tac toe in python*”
2. Prompt: “*in welke directory staat het bestand*?”
3. Start het spel nu vanaf de powershell command line: python `tic_tac_toe.py`
4. Prompt: “*leuk, maar ik wil liever tegen de computer spelen*”
5. Prompt: “*nice! nu een nieuw spel, liever iets uit de lucht schieten spelletje of zo*”
6. Prompt: “*wat betekenen die cijfers (+ en - in kleur groen en rood) tijdens het genereren*”
7. Bekijk hoeveel tegoed je nog hebt.



4: Loganalyse. Gebruik Codex

Wat gaan nuttige zaken halen uit logging. Voor beheerders is het niet te doen om alle logfiles uit te spitten. Het is beter om hier AI voor in te zetten! Moet je natuurlijk wel eerst een logfile hebben. “Gratis” logfiles kan je hier vinden: https://github.com/logpai/loghub Voor dit lab gebruiken we de logging van de SSH service: https://github.com/logpai/loghub/blob/master/OpenSSH/OpenSSH_2k.log

Stappenplan:

1. Heb je de logging gedownload? Zet het in dezelfde directory als waar je Python script staat en gebruik de naam: “OpenSSH_2k.log”
2. Vraag aan AI of deze code kan genereren zodat je alle unieke gebruikers kan halen uit het logbestand. Je mag de URL naar de logfile gebruiken in je vraag **(bewaar het antwoord).**
3. Graag ook de users die “invalid” zijn opnemen. Trouwens, wat is user “0” eigenlijk op ieder linux systeem?
4. Graag de totaal aantallen van valid + invalid **(bewaar het antwoord).**
5. Nieuwe code genereren: vanaf welk ip adres wordt het meeste ingelogd en welke users zijn het die inloggen vanaf dat ip adres? **(bewaar het antwoord).**
6. Hoeveel Chat berichten mag je nog stellen aan Copilot? Zie icoon rechtsonder in VSC **(bewaar het antwoord).**

Ten slotte: Tja, heel veel werk was dit niet, VSC en Copilot hebben het werk gedaan. Lever nu de uitvoer van de opdrachten in, overal waar stond “**bewaar het antwoord**”.

5: Pingtool maken. Gebruik Codex en Powershell.

Maak een tool die is geschreven in PowerShell, die een GUI oplevert (geen command line dus!) en waar je een hostname kan invoeren en waar vervolgens een ping wordt verstuurd naar de host. De uitvoer moet zichtbaar zijn in de tool.

6: Beheertool maken. Gebruik Codex en Python.

Maak nu een beheertool. Deze moet geschreven zijn in python en een hoofdmenu bevatten met daarin 5 opties. De eerste drie opties staan hieronder beschreven. De laatste twee opties verzin je zelf.

1. Pingen van een bepaalde host (heb je al gemaakt, nu in python)
2. Port scanner: Maak een port scanner om uit te vinden of poort 22 (ssh) open staat voor een gegeven ip-adres.
3. IP scanner: Maak een ip scanner die op een heel subnet kan zoeken naar hosts die “up” zijn.
4. Zelf bedenken.
5. Zelf bedenken.

2.7.6 Lesstof tentamen

Wat moet je weten voor het tentamen over dit onderwerp, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie over zowel XML als over Encoding.
- **Lees** de informatie die wordt aangeboden onder het kopje “Informatiebronnen”.
- **Zorg dat je een fout kan halen** uit de Python code, bv. waar ChatGPT het verkeerd heeft gedaan.
- **Zorg dat je een fout kan halen** uit de Python code waarin een XML wordt gelezen.
- Zorg dat je zelf XML-parsing-scripts hebt gemaakt zodat je **getoonde code van anderen goed begrijpt** en kan **aanpassen/verbeteren**.

2.7.7 Informatiebronnen

- XML v1.0.ppt (W3Schools)
- Test je kennis op: http://www.w3schools.com/xml/xml_quiz.asp
- http://www.diveintopython3.net/xml.html , maar niet: “namespaces”, ook niet lxml
- Meer info: https://docs.python.org/3/library/xml.etree.elementtree.html
- Ja hoe kan het ook anders: ChatGPT en Copilot.

2.8 Tooling 1: 'Bare Metal' → OS Deployment

Er bestaan verschillende methoden om hardware te voorzien van een besturingssysteem: Met "bootable media", zoals een installatie CD, in een Cloud omgeving gebruik maken van een template, via multicast een hele groep computers deployen met software zoals "Symantec Ghost" etc. De uitdaging bij deployment is altijd om flexibel te blijven zodat je niet direct een nieuwe image hoeft te maken als je een extra pakket wilt installeren bijvoorbeeld.
In deze week ga je ervaring opdoen met een "Zero Touch" Installatiemethode met behulp van de Preboot Execution Environment (PXE) in een Linux omgeving. Hierbij wordt de installatie over het netwerk gestart.
Deze methode is ook van toepassing op het Windows platform. De gebruikte software heet dan "WDS", "WAIK", "WDT", maar het principe is min of meer gelijk: Boot over het netwerk, laad vervolgens een klein besturingsysteem waarmee je de installatie kan uitvoeren van het eigenlijke besturingssysteem aan de hand van een bestand waarin alle antwoorden staan op de vragen die je anders tijdens een handmatige installatie zou hebben beantwoord. Tegenwoordig worden veel Windows clients geïnstalleerd via Microsoft Intune + Autopilot (cloud gebaseerd), maar dat kunnen we in aan labomgeving niet makkelijk oefenen en bovendien: de principes zijn gelijk aan de andere methodes.
In een volgende weekopdracht gaan we ervoor zorgen dat we het systeem na de eenmalige installatie kunnen configureren en up-to-date kunnen houden via "configuratie management software".

2.8.1 Lesopdracht

Welke methoden ken je om een systeem (fysiek, dus "bare metal", of virtueel in een private of public cloud) te voorzien van een besturingssysteem? Maak in tweetallen een lijstje van de methode en bijbehorende voor- en nadelen.

2.8.2 Weekopdracht

We gaan een "zero Touch" installatie uitvoeren van het Rocky besturingssysteem, dat wil zeggen: "Het systeem wordt geboot waarna het volledig automatisch wordt voorzien van een besturingssysteem, zonder handmatige stappen".
We maken hierbij gebruik van VMware workstation en een tweetal systemen: Het eerste systeem bevat de software om het tweede systeem te kunnen uitrollen.

Rocky 9 (Rocky 9.<latest>, bv 9.8)

Download de laatste Rocky versie op https://rockylinux.org/download of kies een mirror als het traag gaat. Check evt. wel de checksums (vraag je docent hoe dat moet, of google even).
Je kan kiezen voor DVD of Boot. Het is handig om de DVD versie te downloaden, zodat je geen Internet toegang nodig hebt bij de installatie. Dat is wel een grote file (>10GB).

De client (client.school.test)

We beginnen met het bouwen van het tweede linux systeem. Dit wordt het "kale" systeem met alleen een harddisk en ten minste (!) 4 GB memory (liever te veel dan te weinig). Maak hiervoor een nieuwe VM. Paar aanwijzingen hierbij:

- "I will install the operating system later".
- Linux, Rocky Linux 64-bit
- Create a new virtual disk.

- Netwerk: VMnet3 (of ander vmnet, Host-only).- Zorg ervoor dat er geen VMWare DHCP-server actief is op dit subnet. We maken de DHCP service namelijk zelf.
- Finish!

Maak hiervan direct een snapshot met de naam "*bare metal*". Dat test gemakkelijk. De verdere installatie gaan we straks geautomatiseerd uitvoeren. Voor nu even klaar met dit systeem. Eigenlijk heb je nu een lege, "kale" server.

De Linux server vanwaar we alles installeren (salt.school.test)

Nu de installatie van de eerste Linux server vanwaar we alles doen. Hieronder wordt de configuratie toegelicht ("echte" installatie volgt later op deze pagina).

Het eerste Rocky systeem bevat de software waarmee deployments kunnen worden uitgevoerd:

DNS: Zodat de nieuwe systemen de server kunnen vinden.
DHCP: Om een ip-adres te krijgen en het adres van de TFTP server door te geven.
TFTP: Om de Linux kernel te downloaden waarmee de installatie van het echte besturingsysteem kan starten.
FTP: Om het besturingssysteem te downloaden.
Kickstart: Om het besturingssysteem te configureren.

Installeer nu eerst Rocky met de DVD-versie, bijvoorbeeld: Rocky-9-[latest]-x86_64-dvd.iso . Tips hiervoor:

- Software: "**Minimal install**" with "**standard**" **additional software**.
- Create a user and make him/her Administrator
- Netwerkkaart in "NAT".
- "local media" als installatie source.
- Begin installation!

OS geïnstalleerd? We gaan nu het systeem voorbereiden en vervolgens de verschillende softwarecomponenten installeren en configureren. De kennis die je al hebt opgedaan tijdens eerdere cursussen (Basis ICT-Platformen) zoals starten en installeren van services en kennis van nano, dns en dhcp wordt als bekend verondersteld.

```
>>> hostnamectl set-hostname salt.school.test
```

Om de inrichting van de server wat te vergemakkelijken zetten we de firewall en SELinux ('disabled' of 'permissive') uit. Dat doe je natuurlijk niet in een productieomgeving!

```
>>> dnf update
>>> setenforce 0
```

SELinux staat daardoor uit, maar gaat na reboot weer aan, dus ook de regel `SELINUX=enforcing` veranderen naar `SELINUX=permissive`. Dat kan geautomatiseerd met hulp van het onderstaande commando:

```
>>> sed -i 's/^SELINUX=enforcing$/#&\nSELINUX=permissive/' /etc/selinux/config
```

Check natuurlijk wel in `/etc/selinux/config` of het nu goed staat!

Geef dan je server een **tweede** netwerkkaart in een vmnet. Zorg voor deze instellingen:

Interface	**Boot protocol**	**VMnet**
ens160 (1e NIC)	DHCP	NAT
Ens224 (2e NIC)	Static, ip: 192.168.3.100	Vmnet3 (mag ook een ander zijn, maar wel gelijk aan je client)

Via VMnet3 gaan we straks je client connecten, die kan het Internet bereiken via deze server, dus moeten we nog "NAT" en routing regelen, dat kan onder linux heel makkelijk met:

- `firewall-cmd --permanent --change-interface=ens160 --zone=external`
- `firewall-cmd --permanent --change-interface=ens224 --zone=internal`
- `firewall-cmd --set-default-zone=internal`
- `# regels hieronder speciaal voor Rocky>=9`
- `firewall-cmd --permanent --new-policy allowForward`
- `firewall-cmd --permanent --policy allowForward --set-target ACCEPT`
- `firewall-cmd --permanent --policy allowForward --add-ingress-zone internal`
- `firewall-cmd --permanent --policy allowForward --add-egress-zone external`
- `firewall-cmd --reload`

Bekijk de firewall instellingen nu maar eens via onderstaande commando's en let op "`masquerade`". Welke interface doet nu de "NAT"? Op welke interface[s] staat SSH open?

- `firewall-cmd --list-all --zone=internal`
- `firewall-cmd --list-all --zone=external`

DNS:

Je nieuwe client moet straks je server kunnen vinden. Nu minder belangrijk, maar wel noodzakelijk voor de volgende lessen. Kortom, we hebben DNS nodig.

```
>>> dnf makecache
>>> dnf install bind bind-utils
```

Maak een nieuwe zone aan: "school.test". (bestandsnaam: `/var/named/school.test.db`)

Zorg ervoor dat er tenminste een entry wordt aangemaakt voor:

- salt.school.test → 192.168.3.100
- client.school.test → 192.168.3.207

Pas het bestand `/etc/named.conf` aan zodat het allemaal werkt.

Test de configuratie van de DNS voordat je verder gaat:

```
>>> named-checkconf /etc/named.conf
>>> named-checkzone school.test /var/named/school.test.db
```

Start en enable nu je name daemon (named).

Zorg ervoor dat je server nu je eigen DNS server gaat gebruiken ipv de queries forward naar VMWare. Dat kan door de dns-server "127.0.0.1" en "`Ignore automatically obtained DNS parameters`" toe te voegen met nmtui.

Je kan ook de interface files /etc/NetworkManager/system-connections rechtstreeks aanpassen door in `[ipv4]` toe te voegen:

```
dns=127.0.0.1;
ignore-auto-dns=true
```

Werkt dit nu goed? (staat het IP-adres in de 'answer section' bij het onderstaande command?)

```
>>> dig client.school.test
>>> dig www.klm.nl
```

Werkt het niet? Wat staat er in `/etc/resolv.conf` ? Wordt er wel verwezen naar jouw dns server?

Staat de firewall open voor toegang op poort 53 (udp/tcp) vanuit vmnet3?

DHCP

Met Dynamic Host *Configuration* Protocol (DHCP) kan je je nieuwe host configureren met allerlei opties. Uit de cursus basis platformen weet je dat dit een router kan zijn, een time server of een DNS server. We maken nu gebruik van een nieuwe optie, namelijk de optie “next server”en “filename” waarmee je het IP adres en bestandsnaam doorgeeft van de TFTP-server waar de host de PXE-linux omgeving kan downloaden en starten: Een mini-os dus. Met dit mini-os kan via FTP de echte installatie worden gestart.

```
>>> dnf install dhcp-server
>>> nano /etc/dhcp/dhcpd.conf
       ddns-update-style interim;
       ignore client-updates;
       authoritative;
       allow booting;
       allow bootp;
       allow unknown-clients;

       # A slightly different configuration for an internal subnet.
       subnet 192.168.3.0 netmask 255.255.255.0 {
         range 192.168.3.200 192.168.3.254;
         option domain-name-servers 192.168.3.100;
         option routers 192.168.3.100;
         default-lease-time 7200;
         max-lease-time 14400;
         next-server 192.168.3.100; #  PXE/TFTP server IP
         filename "pxelinux.0";
       }
```

Misschien wil je voor je tweede linux server, de client, alvast een reservering aanmaken. Dan krijgt je client altijd hetzelfde ip-adres. Hieronder een voorbeeld van zo’n reservering die je in /etc/dhcp/dhcpd.conf kunt toevoegen. Je mag het ook later toevoegen.

```
  next-server 192.168.3.100; #  DHCP server ip
  filename "pxelinux.0";
}

# Reservation als voorbeeld!
host client {
  hardware ethernet 00:50:56:32:35:B7;
  fixed-address 192.168.3.207;
}
```

TFTP

De PXE ROM maakt gebruik van UDP/IP (voor transport), DHCP (voor configuratie) en TFTP (voor bestandsoverdracht). Trivial File Transfer Protocol (TFTP) maakt gebruik van UDP en is een protocol dat veel gebruikt wordt om computers vanaf een netwerk te starten. TFTP ken je misschien doordat je het ook kan gebruiken om andere apparatuur zoals routers, switches, ADSL- en kabelmodems van firmware en configuraties te voorzien. TFTP werd voor het eerst gedefinieerd in 1980. Door de eenvoud van dit protocol zijn clients en servers relatief eenvoudig te implementeren in kleine hoeveelheden geheugen.
We gaan nu een TFTP daemon installeren en de juiste bestanden kopiëren. Die bestanden komen “gewoon” vanaf de Rocky iso.

```
>>> dnf install tftp tftp-server syslinux wget
```

Dit is dit nog niet genoeg. Je moet ook het bestand tftp.service aanpassen:

```
>>> nano /usr/lib/systemd/system/tftp.service
      [Unit]
      Description=Tftp Server
      Requires=tftp.socket
      Documentation=man:in.tftpd

      [Service]
      ExecStart=/usr/sbin/in.tftpd -s /tftpboot
      StandardInput=socket

      [Install]
      Also=tftp.socket
      WantedBy=multi-user.target
```

Zorg nu voor de juiste data in je /tftpboot directory. Deze benodigde data heb je al verkregen via het pakket "syslinux":

```
>>> mkdir -p /tftpboot/netboot/
>>> mkdir /tftpboot/pxelinux.cfg
>>> cp -v /usr/share/syslinux/pxelinux.0 /tftpboot
>>> cp -v /usr/share/syslinux/menu.c32 /tftpboot
>>> cp -v /usr/share/syslinux/memdisk /tftpboot
>>> cp -v /usr/share/syslinux/mboot.c32 /tftpboot
>>> cp -v /usr/share/syslinux/chain.c32 /tftpboot
>>> cp -v /usr/share/syslinux/ldlinux.c32 /tftpboot
>>> cp -v /usr/share/syslinux/libutil.c32 /tftpboot
```

Starten maar!

```
>>> systemctl enable tftp
>>> systemctl start tftp
>>> firewall-cmd --zone=internal --add-service=tftp --permanent
>>> firewall-cmd --reload
```

Test je of de server werkt door een file op te halen. Dat gan met de tool "tftp" via:

```
>>> cd /tmp/  ;# veilige locatie
>>> tftp 127.0.0.1 -c get memdisk
```

Zie je de file 'memdisk' nu op disk staan? Zo niet, herstart je server en probeer het opnieuw.

PXE Menu

Als het nieuwe systeem (client.school.test) straks boot ziet de gebruiker een menu. We doen zero touch installation, dus de gebruiker hoeft niet te kiezen, maar de gebruiker wil wel zien wat hij/zij heeft gekozen. Kijk goed naar de laatste regels in het bestand. Snap je wat ze betekenen? Let op, **de rode regel is één enkele regel**!

```
>>> nano /tftpboot/pxelinux.cfg/default
      default menu.c32
      prompt 0
      timeout 30
      MENU TITLE HU.School.Test PXE Menu
```

```
LABEL ROCKY9_x64
MENU LABEL ROCKY 9 X64
KERNEL /netboot/vmlinuz
APPEND  initrd=/netboot/initrd.img  inst.repo=ftp://192.168.3.100/pub/ROCKY
inst.ks=ftp://192.168.3.100/pub/ks.cfg
```

VSFTP

Nu nog de ftp-website voor het transport van de Rocky-9-iso naar de nieuwe client maken.

```
>>> dnf install vsftpd
>>> mkdir /var/ftp/pub/ROCKY
```

Mount de DVD met de RedHat Source op de plaats `/var/ftp/pub/ROCKY` en wel zodanig dat je het volgende ziet:

```
>>> ls /var/ftp/pub/ROCKY/
```

```
[root@salt selinux]# ls /var/ftp/pub/ROCKY/
AppStream  BaseOS  EFI  images  isolinux  LICENSE  media.repo  TRANS.TBL
```

Zorg dat de dvd na reboot ook weer gemount is! Met onderstaande tips kom je er wel uit:

- Met `blkid` vind je het UUID van de dvd-speler als je deze hebt “ge-connect” in VMware.
- Voeg een regel toe met daarin de kolommen: UUID, pad (/var/ftp/pub/ROCKY), filesystem (iso9660), en verder nog een paar attributen (ro,user,auto 0 0)

- Test je configuratie voordat je reboot! Bv met: `mount -a` (als er output is, lees het goed en los het probleem op voor je reboot!)

Je hebt nog een paar bestanden uit de ftp-directory nodig voor het mini-os dat wordt gestart via tftp:

```
>>> cp /var/ftp/pub/ROCKY/images/pxeboot/vmlinuz /tftpboot/netboot/
>>> cp /var/ftp/pub/ROCKY/images/pxeboot/initrd.img /tftpboot/netboot/
```

Ten slotte wil je nog anonymous toegang toestaan tot je ftp site. Dat kan bij Rocky door een aanpassing te maken in de file: `/etc/vsftpd/vsftpd.conf` Daar kom je vast wel uit.

Alles gedaan? Enable en start ten slotte de vsftpd deamon.
Denk ook aan de firewall! Tip: `--add-service=ftp`

Wil je weten of de ftp server goed werkt? Test het even met curl, bijvoorbeeld:
`curl ftp://salt.school.test/pub/ROCKY/media.repo`
krijg je een foutmelding over “PASV”? gewoon doorgaan, misschien werkt ’t toch.
Kan het bestand gedownload worden? Ook na reboot? Ga dan verder.

Kickstart

Bijna klaar! Met deze iso kan je veel verschillende systemen uitrollen: desktop, server, grafisch of tekst. Door middel van een kickstart file kan je daar zelf invulling aangeven.
Hieronder de kickstartfile die we gebruiken tijdens dit lab voor de uitrol van een niet-grafisch systeem (minder installatietijd). Maak het onderstaande kickstart bestand aan:

```
>>> nano /var/ftp/pub/ks.cfg
 # Network information
 network --bootproto=dhcp --device=ens160 --activate --hostname=client.school.test
 # Firewall configuration
 firewall --enabled
 # Reboot after installation
 reboot
 # Use FTP installation media
 url --url="ftp://192.168.3.100/pub/ROCKY"
 # Root password [used here "Pa$$w0rd"]
 rootpw --plaintext Pa$$w0rd
 # Use graphical install
 graphical
 #cmdline
 firstboot --disable
 # System keyboard
 keyboard us
 # System language
 lang en_US
 # SELinux configuration
 selinux --permissive
 # System timezone
 timezone Europe/Amsterdam
 # System bootloader configuration
 bootloader --location=mbr
 clearpart --all --initlabel
 part swap --asprimary --fstype="swap" --size=1024
 part /boot --fstype xfs --size=1024
 part pv.01 --size=1 --grow
 volgroup rootvg01 pv.01
 logvol / --fstype xfs --name=lv01 --vgname=rootvg01 --size=1 --grow

 %packages
 @core
 net-tools
 %end
 %post
 %end
```

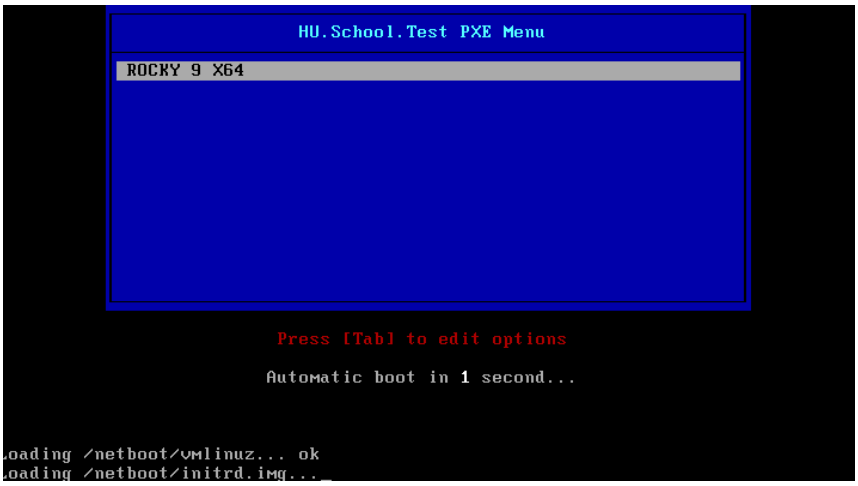
Testen

Draaien dhcpd, tftp, vsftpd en is de iso gemount in /var/ftp/pub? Dan is het tijd om de installatie van de client te testen. Start de client-VM (druk indien nodig op F12)



FIGUUR 1: PXE BOOT MIDDELS DHCP.

Als alles goed is gegaan zie je het onderstaande PXE menu en wordt een linux kernel geladen waarmee de installatie zal beginnen:



FIGUUR 2: BOOTMENU.

Nazorg

Neem na afloop van de installatie de host client.school.test op in de DNS als dat nog niet klopte.
Dit is ook een goed moment om in de dhcp configuratie een reservering voor je client op te nemen als je dat nog niet had gedaan, zie de beschrijving hierboven toen het ging over dhcp.
Maak nu een snapshot van beide systemen. Je hebt ze nodig in een volgende opdracht.

Optioneel (als je het leuk vindt)

Je kan hetzelfde nu bouwen onder Windows: Gebruik hiervoor naast de ISO van een Windows client ook: Windows AD, DNS, DHCP, WDS en eventueel WISM voor het maken van de answer file. Op het Internet bestaan hiervoor veel tutorials…

2.8.3 Lesstof tentamen

Wat moet je weten voor het tentamen over dit onderwerp, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie en dit werkboek over dit onderwerp. De precieze inhoud van de configuratiefiles hoef je niet te kennen. Wel welke services nodig zijn, waar ze voor dienen en hoe je deze globaal configureert.
- **Lees** alle informatie die wordt aangeboden onder het kopje “Informatiebronnen”.

2.8.4 Informatiebronnen

- Meer informatie over tftp: http://www.bo-yang.net/2015/08/31/RHEL7-install-tftp-server

2.9 Tooling 2: Configuration Management Using SaltStack

De servers zijn geïnstalleerd, maar wat nu?! Natuurlijk wil je extra software installeren, users aanmaken, services starten etc. Hiervoor maken we gebruik van “configuration management” (CM) software. Tijdens deze cursus maken we gebruik van SaltStack. In het lab gaan we deze week software installeren en daarna gebruiken om op afstand commando’s uit te voeren op de clients. Het gaat allemaal stap-voor-stap. In de komende week moet je meer zelf gaan uitzoeken. De kennis van deze week is daarbij onmisbaar.

2.9.1 Lesopdracht

Welke Configuration Management software ken jij en wat zijn de verschillen tussen deze tools?

2.9.2 Weekopdracht

Linux1: Master installatie.

De master kan je vergelijken met de “server”, de minions zijn de te beheren systemen, oftewel de “clients”. We installeren eerst de master. Hierbij gaan we uit van het gebruik van een shell met root rechten.

```
# salt zit niet in de base repo, dus voegen we een repository toe
>>> curl -fsSL https://github.com/saltstack/salt-install-
guide/releases/latest/download/salt.repo | sudo tee /etc/yum.repos.d/salt.repo
>>> dnf update

# install salt-master
>>> dnf install salt-master

>>> systemctl enable salt-master
>>> systemctl start salt-master

# zet de firewall open op de nic in de default (=internal) zone
>>> firewall-cmd --permanent --add-service=salt-master
>>> firewall-cmd --reload
```

Linux client: Minion installatie Rocky

Nu de client, ook wel minion genoemd. De installatie is vergelijkbaar met die van de master. Echter, maak je geen zorgen om de firewall. De client heeft alleen uitgaande connecties naar de master en niet andersom.
Voordat je gaat installeren: Heeft je client nu een vast ip adres, bv 192.168.3.207? Je kan de client dan ook toevoegen aan je dns door een entry toe te voegen aan /var/named/school.test.db

Log nu in op de client en test de configuratie voordat je verder gaat: is het internet bereikbaar op je client en kan je de master pingen?

```
>>> ping salt.school.test
>>> ping salt
>>> ping google.nl
```

Zo ja, ga dan verder:

```
# salt zit niet in de base repo, dus voegen we een repository toe
>>> curl -fsSL https://github.com/saltstack/salt-install-
guide/releases/latest/download/salt.repo | sudo tee /etc/yum.repos.d/salt.repo

# nu kan je de salt-software installeren
>>> dnf -y install salt-minion

# pas de '#master: salt'-regel aan in /etc/salt/minion indien nodig (default is al
salt, dat is eigenlijk al goed, salt.school.test is nog beter misschien).
>>> nano /etc/salt/minion
>>> systemctl enable salt-minion
>>> systemctl start salt-minion
```

Nu weer inloggen op de server: Je client heeft via DHCP een IP-adres gekregen en een key opgestuurd naar de master. Deze moet je nog accepteren vanaf de master. Bekijk eerst alle keys met:

```
>>> salt-key -L
```

Note: Geeft dit een foutmelding (python error)? Dat is een recente bug. Je kan doorgaan, lijkt wel te werken, of "auto accept" van alle keys aanzetten in de file `/etc/salt/master` en daarna salt-service herstarten:

```
# Enable auto_accept, this setting will automatically accept all incoming
# public keys from the minions. Note that this is insecure.
auto_accept: True
```

Accepteer nu een specifieke key[1] met `salt-key -a <hostname>` , of accepteer alle keys met:

```
>>> salt-key -A
```

Als je vanaf de master je minion niet kan bereiken moet je deze opnemen in de DNS, waarschijnlijk in /var/named/school.test.db of anders toevoegen via je /etc/hosts file.

Test nu je installatie vanaf de master met:

```
>>> salt '*' test.ping
```

Krijg je een response van de client? Bij een 'True' response is de installatie gelukt.

Derde Linux VM: Minion installatie Rocky met behulp van kickstart

Wel een beetje jammer natuurlijk dat je na een automatische installatie met PXE en kickstart nu handmatig de saltstack-client moet installeren! Dat moet beter kunnen en dat is ook zo. Doe dit door de onderstaande kickstart configuratie toe te voegen aan je **kickstart file**. Denk ook aan de naam van de host, kies voor client.school.test. Verwijder evt. eerst de huidige client.school.test van de master met behulp van: `salt-key -d client.school.test`
Zorg je er wel voor dat de minion straks het Internet kan bereiken?
De commando's worden automatisch uitgevoerd nadat je het OS hebt geïnstalleerd en vervolgens kan je je clients op afstand en automatisch beheren!

[1] Je kunt ook een autosign file maken waarbij je bijvoorbeeld wildcards gebruikt *.school.test Als minions die met deze domeinnaam connectie maken met de master, dan worden deze automatisch geaccepteerd. Handig als je ALLES wilt automatiseren.

Test dan je OS-installatie van een nieuwe Rocky client. Hier de info voor je kickstart file. Curl commando is één lange regel!

```
%post
curl -fsSL https://github.com/saltstack/salt-install-guide/releases/latest/download/salt.repo |
sudo tee /etc/yum.repos.d/salt.repo
dnf install -y salt-minion
sed -i "/#master:/c\master: salt.school.test" /etc/salt/minion
systemctl enable salt-minion
# makkelijk op afstand inloggen (niet zo veilig)
echo "PermitRootLogin yes" >> /etc/ssh/sshd_config
# alvast handig voor de volgende les
salt-pip install PyMySQL
%end
```

Denk ook weer aan het accepteren van de key, de namenresolutie en het testen via `salt '*' test.ping`

Windows1: Minion installatie voor Windows

Je kan van veel verschillende besturingssystemen een Saltstack client maken. Dat geldt ook voor MS Windows Server 2025. Hieronder de stappen die nodig zijn voor de -handmatige- installatie:

- Installeer een Windows 2025 Server of maak gebruik van een VM die je nog hebt van een eerdere cursus.
- Kies voor de "Desktop Experience", dat werkt nu het gemakkelijkst.
- Zelfde VMnet (VMnet 3?) gebruiken.
- Zorg ervoor dat je server een vast ip adres heeft in het 192.168.3.0/24 netwerk, bv 192.168.3.205. Denk ook aan je default route naar de master en laat ook de DNS via de master verlopen.
- Als het goed is kan je naar www.google.nl met IE en krijg je response als je pingt naar de master:

```
PS C:\Users\Administrator> ping salt.school.test

Pinging salt.school.test [192.168.3.100] with 32 bytes of data:
Reply from 192.168.3.100: bytes=32 time<1ms TTL=64
Reply from 192.168.3.100: bytes=32 time<1ms TTL=64
```

- Download nu de SaltStack minion software: https://docs.saltproject.io/salt/install-guide/en/latest/topics/install-by-operating-system/windows.html . In combinatie met Rocky9.8 is het verstandig om dezelfde versie van Salt te downloaden (=3008 LTS, AMD64 MSI).
- Installeer de software (master: "salt.school.test", minion name: "`windows1.school.test`").
- Configuratie en logging staat in: c:\programdata\salt project\salt*
- Accepteer de key op de master net zoals je deed bij de Linux client.
- Is je windows server ook toegevoegd aan de DNS op Linux?
- Test je clients met: `salt '*' test.ping`
- Werkt het niet, kijk dan `in C:\ProgramData\Salt Project\Salt\` voor logging en confiugratie.

Remote Execution

Je kan nu op afstand commando's uitvoeren. Voer de onderstaande commando's uit en schrijf op wat deze eigenlijk doen. Niet alles werkt goed, snap je waarom sommige commando's niet zo goed werken op alle minions?

```
# ping
salt windows1.school.test test.ping
```




```
salt '*' test.ping

# remote command
salt client.school.test cmd.run pwd
salt '*' cmd.run pwd   # niet alles wordt ondersteund onder windows

# Disk Usage
salt '*' disk.usage

# Permissions
salt client.school.test acl.getfacl /root/

# Status of all minions
salt-run manage.status   # negeer evt. Python errors, bug in deze versie

# grains, list all items
salt '*' grains.ls

# Grain(s)
salt '*' grains.get ip4_interfaces
salt 'windows1.school.test' grains.get osfullname
salt 'windows1.school.test' grains.items
salt '*' grains.item saltversion os_family
```

Configuration Management

Mooi, we kunnen onze minions op afstand “besturen”. Door gebruik te maken van state-files kan je ervoor zorgen dat je in een bestand kan opgeven hoe je minion geconfigureerd is. Als voorbeeld gaan we ervoor zorgen dat je een python script plaatst op je minion en deze vervolgens uitvoert op de minion.

Default verwacht de master je State-files en andere bestanden onder `/srv/salt/` , maar dat kan je veranderen in `/etc/salt/master`.

Maak nu eerst een bestand /srv/salt/scriptje.py aan op de master met de onderstaande inhoud:

```
#!/bin/python3
f=open('output.log', 'a')
f.write('Hello world\n')
f.close()
```

Snap je wat hierboven gebeurt? Waar wordt output.log geplaatst denk je als je het start?

We gaan de acties “bestand kopiëren” en “bestand runnen” beschrijven via YAML in het bestand /srv/salt/example.sls. Dat heet een “state-file”:

```
mijnpython:
  pkg.installed:
    - name: python3
  file.managed:
    - name: /tmp/scriptje.py
    - source: salt://scriptje.py
    - mode: 777
```

```
    cmd.run:
      - name: /tmp/scriptje.py
      - cwd: /tmp/
```

Snap je wat hierboven gebeurt? (`salt://` verwijst trouwens naar de root van je bestanden, default `/srv/salt/`)

Je kan de goede werking nu testen vanaf de master met het commando (let op, niet de extensie ".sls") gebruiken bij de uitvoer:

```
>>> salt client.school.test state.apply example
```

Wat gebeurt er als je het bestand /tmp/scriptje.py verwijdert op de minion en je doet dit commando nog een keer?

Bijna klaar! In het bovenstaande commando geef je heel expliciet aan welke state-file je wilt toepassen op welke minion. Als je veel state files hebt is dat niet handig. Je kan een bestand maken, top.sls waarin je verwijst naar alle onderliggende state-bestanden. Door vervolgens top.sls uit te voeren worden alle configuraties gechecked en uitgevoerd:

Maak het bestand /srv/salt/top.sls aan:

```
base:
  '*':
    - example
```

Hier staat eigenlijk: *voer op alle minions in de base-environment de "state" door zoals omschreven in het bestand example**.sls***
Je kan dit doorvoeren op client.school.test met het onderstaande commando waarin je de state-file niet noemt:

```
>>> salt client.school.test state.apply
```

Ten slotte willen we er nog voor zorgen dat deze configuratie alleen wordt uitgevoerd op de Rocky systemen. Voer het onderstaande commando uit:

```
>>> salt '*' grains.get os
```

Je leert van de uitvoer van dit commando dat de linux systemen van het os "Rocky" zijn. Verander nu de top.sls naar:

```
base:
  'G@os:Rocky':
    - example
```

En run `salt '*' state.apply`. De foutmelding kan je negeren: er wordt alleen aangegeven dat er voor het windows1 systeem geen state-files zijn gevonden en dat klopt.

Kan je er nu voor zorgen dat het script ook onder windows wordt uitgevoerd?
Je mag gebruik maken van de onderstaande informatie:

```
python_run_script:
  file.managed:
    - name: 'c:\temp\scriptje.py'
    - source: 'salt://scriptje.py'
    - makedirs: true
  cmd.run:
    - name: '"C:\Program Files\Salt Project\Salt\Scripts\Python.exe" c:\temp\scriptje.py'
    - cwd: 'c:\temp'
```

Denk ook aan het aanpassen van de top.sls file.

We zijn nu klaar: je kan met 1 commando (salt '*' state.apply) alle state files activeren op je minions.

2.9.3 Lesstof tentamen

Wat moet je weten voor het tentamen over dit onderwerp, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie en dit werkboek over dit onderwerp. De precieze inhoud van de configuratiefiles hoef je niet te kennen. Wel welke services nodig zijn, waar ze voor dienen en hoe je deze globaal configureert.
- **Leer** de theorie over SaltStack zoals te vinden op Canvas over dit onderwerp.

2.9.4 Informatiebronnen

- Tutorial SaltStack: https://blog.woohoosvcs.com/2019/11/introduction-to-saltstack/

2.10 Tooling 3: Configuration Management (NextCloud)

Vorige week heb je ervaring opgedaan met de basisprincipes van SaltStack. Deze week gaan we een stap verder. Op het Internet staan veel “tutorials”, “how to’s” en “filmpjes” waarin de configuratie van software wordt uitgelegd. Zou het niet veel handiger zijn als je een paar SaltStack files hebt voor jouw platform waarna de hele installatie en configuratie wordt uitgevoerd? Aan de hand van de YAML-files kan je ook precies zien wat er gebeurt, kortom een werkende omgeving en ook nog gedocumenteerd!
Door middel van deze weekopdracht doe je de noodzakelijke kennis op die je nodig hebt voor de eindopdracht.

2.10.1 Lesopdracht

--

2.10.2 Weekopdracht

Tijdens deze weekopdracht ga je de software “**NextCloud**” installeren op het Rocky platform. Gebruik hiervoor je **client**, dat is het gemakkelijkst. Misschien moet je **server** ook wel aanstaan i.v.m. de DHCP.
NextCloud is een vervanging voor GoogleDrive, Dropbox en Onedrive zonder mogelijke “pottenkijkers” zou je kunnen zeggen.

Je installeert aan de hand van een tutorial die je “omzet” naar Saltstack configuratie. De NextCloud server gebruiken we nog in een volgende les.

Het uiteindelijke doel is dat je een ingerichte NextCloud server kan installeren op het Rocky platform zonder dat je daarvoor handmatige acties moet uitvoeren. Zorg dat je in je configuratie gebruik maakt van: YAML, Grains, State files en Pillars.

Zet de onderstaande tutorial om naar een automatische Saltstack configuratie als vervolgstap op de installatie uit de vorige les…

Bron: https://docs.nextcloud.com/server/latest/admin_manual/installation/example_centos.html
Paar opmerkingen hierbij:

- Een eventuele presentatie in de les is bedoeld om je een beetje op weg te helpen. Daarmee wordt lang niet alles duidelijk, het is de bedoeling dat je zelf gaat puzzelen.
- Maak een snapshot van je client,
- **Tip: Installeer het eerst handmatig. Werkt ‘t? Ga dan alle stappen automatiseren met saltstack, steeds 1 stap erbij.**
- **Tip2**: Maak verschillende saltstack files en test ze individueel, bijvoorbeeld: state-file voor apache met php, state-file voor mariadb en een state-file voor NextCloud.
- **Tip3**: Werkt ‘t? Maak het dan “mooi” door pillars toe te voegen, bijvoorbeeld voor database naam, user, password.
- **Tip4**: Niet verplicht, maar voor degenen die echt alles willen automatiseren: kijk eens in de directory `/var/www/html/nextcloud` daar zit de file `occ` en als je deze aanroept met de optie `maintenance:install` kan je ook de configuratie van de database automatiseren, dat is een stap van de handmatige setup, zie hieronder.
- **Tip5**: Denk ook aan “include” om ervoor te zorgen dat de verschillende statefiles worden verwerkt, bv: nextcloud state file kan de statefiles van de database en apache “includen”.

- **Tip6**: Denk ook aan “require” om ervoor te zorgen dat een module eerder wordt uitgevoerd dan een andere module.

Benodigde packages:	`dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-8.noarch.rpm`
	`dnf install -y epel-release yum-utils unzip curl wget \ bash-completion policycoreutils-python-utils mlocate bzip2`
Apache	`dnf install -y httpd`
	Maak een virtual host in /etc/httpd/conf.d/nextcloud.conf met daarin als informatie:

	`<VirtualHost *:80>`
	`  DocumentRoot /var/www/html/nextcloud/`
	`  ServerName  client.school.test`
	
	`  <Directory /var/www/html/nextcloud/>`
	`  Require all granted`
	`  AllowOverride All`
	`  Options FollowSymLinks MultiViews`
	
	`  <IfModule mod_dav.c>`
	`  Dav off`
	`  </IfModule>`
	
	`  </Directory>`
	`</VirtualHost>`

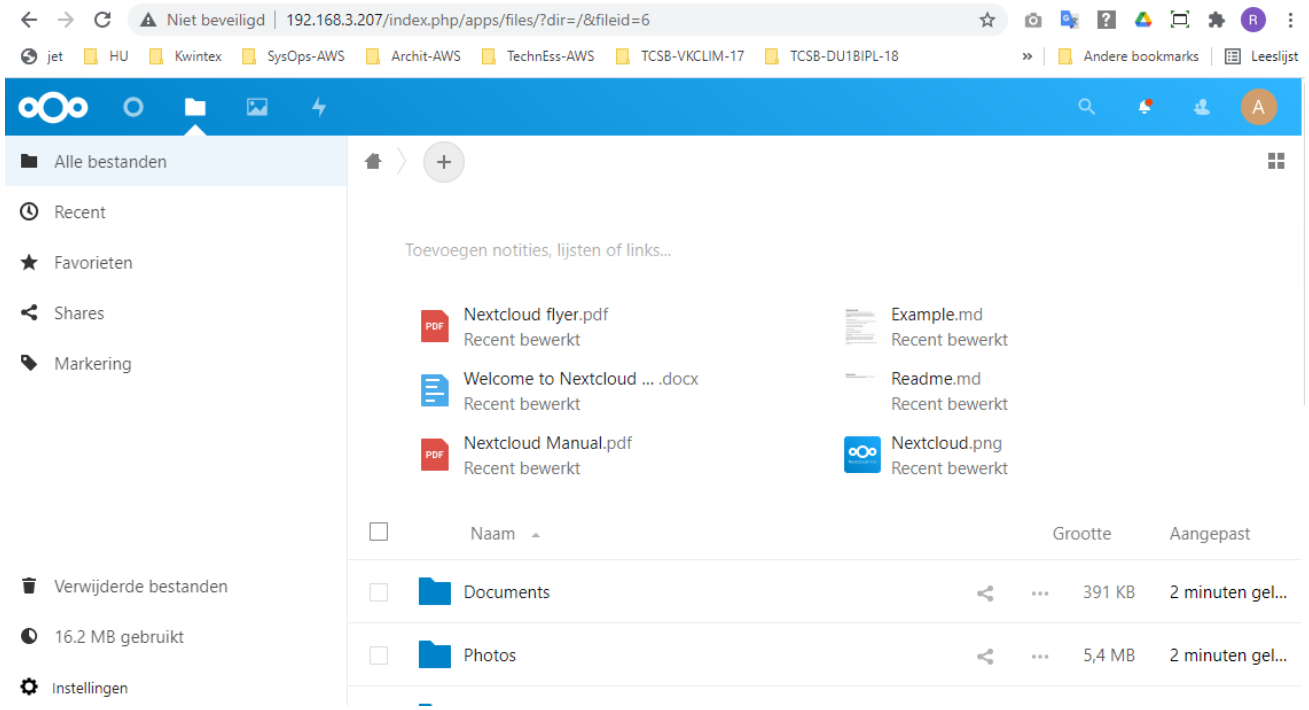
	`systemctl enable httpd.service` `systemctl start httpd.service`
	`firewall-cmd --add-service=http --permanent` `firewall-cmd --reload`
PHP	`dnf module enable php:8.2`
	`dnf install -y php php-gd php-intl php-pecl-apcu php-mysqlnd php-pecl-zip`
Mariadb/Mysql	`dnf install -y mariadb mariadb-server python3-PyMySQL`
	`systemctl enable mariadb.service` `systemctl start mariadb.service`

Maak je database	`mysql -u root -p`
	
	`create database nextcloud;`
	`grant all on nextcloud.* to 'dbuser'@'localhost' identified by '<<hierjewachtwoord>>';`
	`FLUSH PRIVILEGES;`
	`exit`

Redis (voor caching)	`dnf install -y redis`

	systemctl enable redis.service systemctl start redis.service
NextCloud	Download de Server software (Download for server) van https://nextcloud.com/install/#instructions-server Kies voor “download server” -> “community projects”-> Archive -> Download .tar.bz2 Kies voor het ‘.tar.bz2’-archief, voorstel: https://download.nextcloud.com/server/releases/nextcloud-32.0.0.tar.bz2
Uitpakken in de juiste dir	tar -xjvf nextcloud-32.0.0.tar.bz2 -C /var/www/html/
	mkdir /var/www/html/nextcloud/data
	chown -R apache:apache /var/www/html/nextcloud
	systemctl restart httpd.service
we’ll set SELinux. Many tutorials will tell you to disable SELinux (because it is a difficult component to manage). Instead, I suggest you to keep it on and add the rules for NextCloud:	semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/data(/.*)?' semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/config(/.*)?' semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/apps(/.*)?' semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/.htaccess' semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/.user.ini' semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/3rdparty/aws/aws-sdk-php/src/data/logs(/.*)?' restorecon -R '/var/www/html/nextcloud/' setsebool -P httpd_can_network_connect on
Open firefox	http://192.168.3.207/
Choose Mariadb/MySQL	Maak een beheerders account Stel de database in, niet SQLite! Weet je het ww nog van dbuser? (zie hierboven). Krijg je een foutmelding: let op de url, ga naar 192.168.3.207 Helpt dat?

De onderstaande stappen **hoef je niet te automatiseren**, maar voer je uit vanaf de webbrowser:
Test nu of de omgeving “werkt” door te browsen naar: http://192.168.3.207/ en kijk of je bij je bestanden kan komen, bv:



Zo, dat was handmatig, maar nu met saltstack, volledig geautomatiseerd. Hoe dat moet moeten jullie zelf uitzoeken. In ieder geval met state-files en pillars. Hieronder wat tips:

COMMAND	TIP
`dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-8.noarch.rpm`	**pkg.installed , gebruik daarbij de parameter sources**
`dnf install -y epel-release yum-utils unzip curl wget bash-completion policycoreutils-python-utils mlocate bzip2`	
`dnf install -y httpd`	
`Maak een virtual host in /etc/httpd/conf.d/nextcloud.conf met daarin als informatie:`	

`<VirtualHost *:80>`
`  DocumentRoot /var/www/html/nextcloud/`
`  ServerName  client.school.test`
 	
`  <Directory /var/www/html/nextcloud/>`
`  Require all granted`
`  AllowOverride All`
`  Options FollowSymLinks MultiViews`
 	
`  <IfModule mod_dav.c>`
`  Dav off`
`  </IfModule>`
 	
`  </Directory>`
`</VirtualHost>`
	file.managed
	
	zet informatie hiernaast in een file bij je state files

systemctl enable httpd.service systemctl start httpd.service	**service.running**
firewall-cmd --add-service=http --permanent firewall-cmd --reload	**firewalld.present**
dnf module enable php:8.2	**Lukt niet echt anders dan met cmd.run (denk aan -y!)**
dnf install -y php php-gd php-intl php-pecl-apcu php-mysqlnd php-pecl-zip	
dnf install -y mariadb mariadb-server python3-PyMySQL	
systemctl enable mariadb.service systemctl start mariadb.service	
mysql -u root -p create database NextCloud_db; grant all on NextCloud_db.* to 'dbuser'@'localhost' identified by '<<hierjewachtwoord>>'; FLUSH PRIVILEGES; exit	**zet "salt-pip install PyMySQL" onderaan in de ks.cfg en maak een nieuwe client.** **Lukt het niet met salt modules, overweeg dan cmd.run waarin je mysql commando's uitvoert.** **Gebruik hierbij: mysql --protocol=socket --socket=/var/lib/mysql/mysql.sock** **als het werkt: zet password/user/database in een pillar file**
dnf install -y redis systemctl enable redis.service systemctl start redis.service	
Download de Server software (Download for server) van https://nextcloud.com/install Kies voor het '.tar.bz2'-archief, voorstel: https://download.nextcloud.com/server/releases/nextcloud-32.0.0.tar.bz2	**archive.extracted met parameter "skip_verify: True"**
tar -xjvf filename.tar.bz2 -C /var/www/html/	**archive.extracted, zie hierboven**
mkdir /var/www/html/nextcloud/data	
chown -R apache:apache /var/www/html/nextcloud	**archive.extracted, zie hierboven kan allemaal in dezelfde functie**
systemctl restart httpd.service	
semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/data(/.*)?' semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/config(/.*)?'	**selinux.fcontext_policy_present met parameter "sel_type: httpd_sys_rw_content_t"**

semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/apps(/.*)?' semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/.htaccess' semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/.user.ini' semanage fcontext -a -t httpd_sys_rw_content_t '/var/www/html/nextcloud/3rdparty/aws/aws-sdk-php/src/data/logs(/.*)?' restorecon -R '/var/www/html/nextcloud/' setsebool -P httpd_can_network_connect on	**selinux.fcontext_policy_applied** **selinux.boolean**
http://192.168.3.207/	
Maak een beheerders account Stel de MySQL database in, niet SQLite!	

2.10.3 Lesstof tentamen

- **Leren**: Zie 2.9
- **En bovendien**: Powerpoint van deze les.

2.10.4 Informatiebronnen

- Saltstack modules https://docs.saltproject.io/en/latest/ref/states/all/index.html

2.11 Tooling 4: OS Virtualization using Containers

Containers zijn de toekomst voor het effectief beheren van systemen. Het is eigenlijk een volgende stap in virtualisatie. Oorspronkelijk alleen mogelijk onder Linux, maar nu ook mogelijk met behulp van Windows Server. Dit laatste besturingssysteem kent twee typen containers: Windows Server en Hyper-V-containers. Je kan ze beheren met PowerShell of met Docker. Om de basis van containers te oefenen maken we gebruik van een Rocky distributie en Docker voor dit lab.

2.11.1 Lesopdracht

-

2.11.2 Weekopdracht

Stap 1: Installeer Docker op Rocky

Gebruik een linux server (de saltstack master bijvoorbeeld kan je gebruiken, maar dan heb je geen GUI die je nodig hebt bij stap 5. Misschien kan je erom heen werken, anders: gebruik een linux GUI).
We gaan er bij de installatie vanuit dat je dit als “root” uitvoert.

Je moet de docker repository toevoegen zodat je met de package manager “dnf” de docker software kan installeren:

```
tee /etc/yum.repos.d/docker.repo <<-'EOF'
[dockerrepo]
name=Docker Repository
baseurl=https://download.docker.com/linux/centos/9/$basearch/stable
enabled=1
gpgcheck=1
gpgkey=https://download.docker.com/linux/centos/gpg
EOF
```

Installeer de packages en vergeet niet om daarna de `docker.service` te enablen en te starten:

```
dnf install docker-ce docker-ce-cli
```

Mocht dit niet lukken, gebruik het volgende commando:

```
dnf --allowerasing install docker-ce docker-ce-cli
```

Stap 2: Beheren van Docker

Het beheer van docker gaat prima onder je eigen useraccount als je jezelf toevoegt aan de docker group.
Installeer eerst docker, zie hieronder, en doe daarna:

```
usermod -a -G docker <<jouwusername>>
```

Het beheer van docker gaat met... “docker”. Run het commando

```
docker
```

Uitleg krijg je door een specifiek subcommando, type:

```
docker subcommand --help
```

Stap 3: werken met containers

Om een container te kunnen starten, moet je eerst een image downloaden. Deze images kan je downloaden vanuit de Docker Hub, een registry gemanged door Docker, het bedrijf achter "docker". Iedereen kan containers bouwen en hosten op de Docker Hub, dus veel applicaties en linux distributies zijn beschikbaar in deze registry. Controleer of docker goed is geinstalleer door een test image te runnen in de container.

```
docker run hello-world
```

Zie je "Hello from Docker!" in de tekst voorkomen? Dan is je installatie geslaagd. Krijg je een error, misschien even uitloggen en inloggen?

Je kan zoeken naar images op de Docker Hub met het docker commando en search subcommando, bv om centosimages te vinden type je:

```
docker search ubuntu
```

Je ziet allerlei images, maar let op. In de kolom "OFFICIAL" zie je "OK", dat is de image die wordt ondersteund door het bedrijf achter het project. Ook herkenbaar omdat er geen slash in de naam voorkomt: `ubuntu`

Je kan deze image downloaden naar je computer met het pull subcommando:

```
docker pull ubuntu
```

Omdat je geen versie hebt opgegeven krijg je de nieuwste versie (latest). Let op, dit is dus een `Ubuntu container` geïnstalleerd op je host Rocky-besturingssysteem!

Bekijk alle images die je nu hebt gedownload met:

```
docker images
```

Tot nu toe heb je nog niets gedaan met de containers. Je kan er niet eens bij! Daarom, start de `ubuntu` image met de opties -i en -t: je krijgt dan een interactieve shell, bv:

```
docker run -it ubuntu
ps
uname -a
cat /etc/os-release
```

Let op de hostname in de vorm van een nummer. Hierdoor zie je dat je werkt binnen de container.

```
[root@salt ~]# docker run -it ubuntu
root@da7394b6d910:/# ps
    PID TTY          TIME CMD
      1 pts/0    00:00:00 bash
      8 pts/0    00:00:00 ps
root@da7394b6d910:/# uname -a
Linux da7394b6d910 4.18.0-372.26.1.el8_6.x86_64 #1 SMP
```

Vergelijk de linux versie met de versie van je host computer. Zie je dat dit een andere versie is! Je werkt met een Linux systeem in een linux systeem! Vergelijk via uname -a op zowel computer als container ook de kernel versie van beiden. Wat valt je op?

Met exit kan je de interactieve shell verlaten en kom je terug op de host.

```
exit
```

Stap 4: MariaDB gebruiken met behulp van Docker.

Zoek met docker naar de naam van mariadb

```
docker search mariadb
```

Download nu de meest recente mariadb image. Het opgeven van ":latest" is de default en mag je daarom ook weglaten.

```
docker pull mariadb:latest
```

Bekijk je lijst met images met

```
docker images
```

```
[root@salt ~]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
mariadb       latest    14f1097913ec   4 days ago      384MB
ubuntu        latest    216c552ea5ba   5 days ago      77.8MB
hello-world   latest    feb5d9fea6a5   12 months ago   13.3kB
```

Een image is geen running proces, het is alleen de software. Om deze te runnen moet je een container maken.

Het juiste commando hiervoor vind je meestal in de documentatie van de image. Bijvoorbeeld om een container te maken met de naam "mariadbtest" type je het volgende commando:

```
docker run --name mariadbtest -e MYSQL_ROOT_PASSWORD=mypass -d mariadb
```

Met de optie "-e" kan je opties doorgeven voor de mysql-daemon.
Met de optie "-d" draait de container in de achtergrond en wordt je container ID geprint. Elke container heeft zijn id. Docker geeft alle containers een id. Om zeker te weten dat de container "draait" kan je een lijst opvragen met running containers:

```
docker ps
```

Je ziet dan alle "draaiende" containers, wil je alle containers zien type dan:

```
docker ps -a
```

Docker containers kan je pauzeren, stoppen, starten en herstarten.

```
docker pause mariadbtest
docker unpause mariadbtest
docker start mariadbtest
docker restart mariadbtest
docker stop mariadbtest
```

Het troubleshooten van een container? Zorg natuurlijk wel dat de container is gestart.

```
docker logs mariadbtest
```

Wil je interactieve toegang tot de container. Voer dan een interactief commando uit op de container. Zorg natuurlijk wel dat de container is gestart.

```
docker exec -it mariadbtest bash
```

Vanaf nu gaan we verder met de Docker "workshop"zoals Docker dit heeft bedacht:

Stap 5: Containerize an application.

https://docs.docker.com/get-started/tutorials/run-an-app/

paar tips:

- `dnf install git`
- niet alleen commando's uitvoeren, maar probeer te begrijpen wat je doet en wat er gebeurt.
- Zorg dat je een account hebt in de docker hub (https://hub.docker.com/)

Stap 6: Next step: Nu zelf!
Alles duidelijk tot nu toe? Nu iets creatiever. Maak nu een eigen Dockerfile waarbij je een Flask applicatie kan hosten in de container. Je mag daarbij de Flask applicatie gebruiken die je al bij een eerdere opdracht hebt gemaakt tijdens dit vak of een Flask applicatie die je zelf maakt of door AI laat genereren. Zorg ervoor dat je deze applicatie vanaf je eigen laptop browser kan bereiken.

2.11.3 Lesstof tentamen

- **Leer** alle informatie die je vindt in de powerpoint presentatie over dit onderwerp.
- **Leer** de theorie over Docker zoals te vinden op Canvas over dit onderwerp.
- **Optioneel:** de informatie die wordt aangeboden onder het kopje “Informatiebronnen”. Het gaat daarin niet om nieuwe informatie, maar om verbreding zodat je de stof beter begrijpt of kan toepassen.

2.11.4 Informatiebronnen

- “Alle” nuttige docker commando’s en cheat sheet: https://github.com/wsargent/docker-cheat-sheet

2.12 Tooling 5: Test Automation

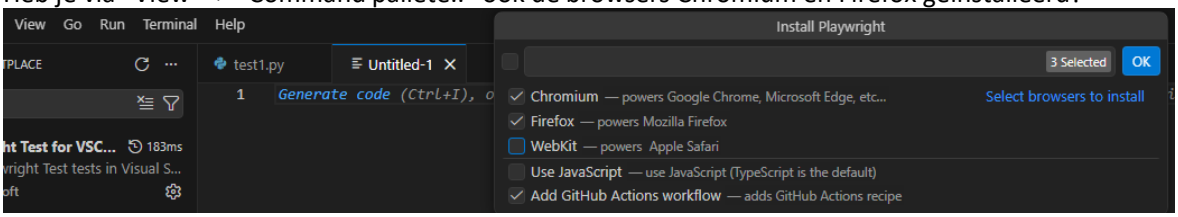
Testen is een gedeelde verantwoordelijkheid: Developers maken nieuwe functionaliteit, maar letten soms minder op non-functionele aspecten zoals security of performance. Beheerders zijn gebaat bij een stabiele, betrouwbare omgeving. Kortom: testen doe je samen en tools/scripts helpen daarbij.
Vroeger kwam er misschien ieder jaar een nieuwe release en kon je de software ook handmatig testen. Tegenwoordig werken we vaak “agile” en dat betekent veel vaker een nieuwe software release. Handmatig testen wordt dan heel vervelend. In deze cursus maak je kennis met “Playwright”, een open source suite aan tools gemaakt door Microsoft waarmee je websites volledig geautomatiseerd kunt testen.

We gaan deze ontwikkelen onder windows. De les is getest met behulp van een VM met daarin Windows Server 2022, Chrome, Firefox en Visual Studio Code. Je kan ook andere keuze maken, bv installatie op je eigen laptop, als ’t straks maar werkt! Je maakt kennis met een nieuwe programmeertaal: TypeScript. Deze taal lijkt sterk op JavaScript, maar voegt typecontrole toe. Daardoor kun je variabelen een type geven, zoals string, number of boolean. Python kent geen typecontrole, veel programmeertalen doen dat wel. Je mag gebruik maken van ChatGPT / Google Gemini of andere chatbot.

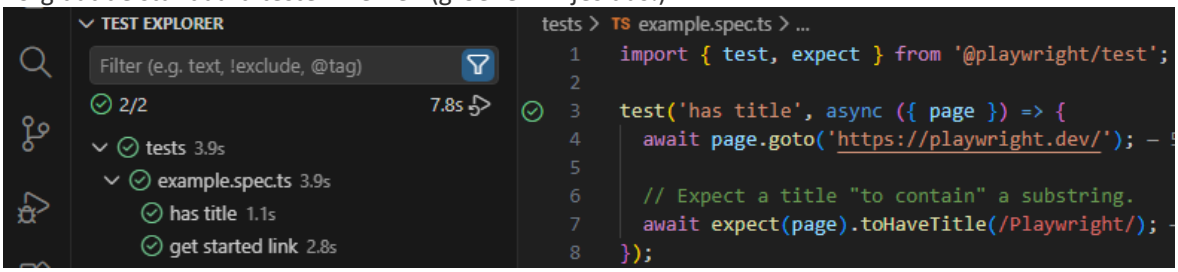
2.12.1 Lesopdracht

We volgen deze handleiding: https://playwright.dev/docs/getting-started-vscode

- Installeer allereerst een nieuwe VM als je je eigen laptop “schoon” wilt houden. Misschien heb je nog een Windows VM die je kan gebruiken?
- Installeer daarna Node.js zodat je Typescript kan “draaien”. Node.js is vooral de runtime omgeving hiervoor. Ik stel voor dat je een “prebuilt Node.js” versie, liefst LTS versie installeert zoals: node-v24.14.0-x64.msi
- Installeer dan Visual Studio Code: https://code.visualstudio.com/
- Volg verder de handleiding https://playwright.dev/docs/getting-started-vscode tot “Debugging Your Tests".
- Dan nu even checken. Heb je dit allemaal gedaan?
    - Dus installeer in visual studio code “Playwright” extensie van Microsoft.
    - Maak een nieuwe folder aan en open die in Visual Studio Code (VSC)!
    - Zet “Autosave” (File -> Autosave) aan in VSC.
    - Heb je via “View” -> “Command pallete..” ook de browsers Chromium en Firefox geïnstalleerd?

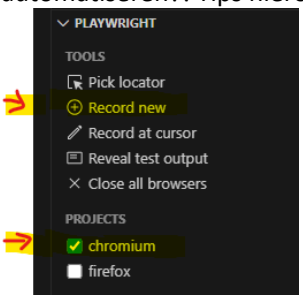


    - Zorg dat de standaard testen werken (groene vinkjes dus!)



Is dat allemaal gelukt? Ga dan nu verder met deze pagina: https://playwright.dev/docs/codegen .
Je kan in playwright zelf alle acties schrijven in TypeScript, maar gebruik je playwright CodeGen, dan klik je gewoon in je browser en kan je navigeren, playwright schrijft zelf de code. De code is wel "TypeScript", geen python helaas.

- Lees de pagina tot "Generate tests with the Playwright Inspector".
- Neem nu zelf een test op: Ga naar www.startpagina.nl en zoek daarin op "fitness". Kan je dat automatiseren?! Tips hieronder.



2.12.2 Weekopdracht

Dit was een eenvoudige test. We maken de test nu moeilijker en proberen deze straks ook uit te voeren vanaf de command line zonder "menselijk interactie", helemaal automatisch dus!

De opdracht bestaat uit verschillende stappen:

1. In de eerste stap installeer je een website in een container, dan hebben we iets om te testen!
2. Die ga je vervolgens gaat testen met behulp van playwright.

Als 't allemaal niet perfect werkt, pas het dan een beetje aan zodat je in ieder geval verder kan.

Stap 1: Installeer een website als linux container op je eerste VM, zodat we iets hebben om te kunnen testen. Gebruik gerust je Linux server uit de vorige les waarmee je docker containers kan draaien. Hier zetten we de website op. Tips:

```
docker run -d -p 3000:3000 --name juice-shop bkimminich/juice-shop
```

Denk je ook even aan de firewall?
Browse vanuit je testserver of laptop naar: http://<ipvanjedockerserver>:3000 → gelukt? Dan ben je klaar om de echt tests te maken.

Stap 2: Record en Test de nieuwe container website met Playwright.

a. **Test1: RegisterUser**
   Maak een test met de naam "RegisterUser" waarin je naar de OWASP Juice Shop gaat en een **nieuw account** aanmaakt.
   Klik op Account
   Klik op Login
   Klik op "Not yet a customer?"
   Registreer je als user door alle velden in te vullen.

b. Test uitbreiden

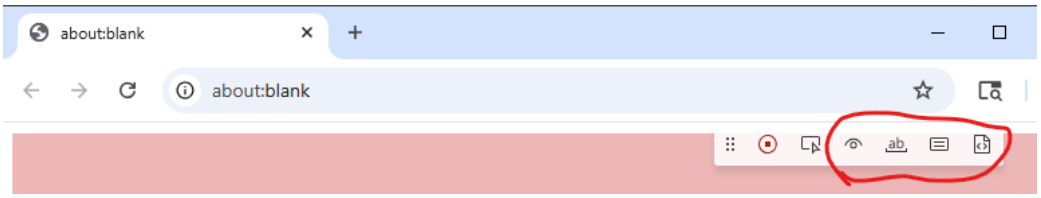
Je zal zien dat je deze test maar 1x goed kan doen, daarna bestaat de naam al! Maak 'm daarom **dynamisch**. Gebruik als Email adres jouw voornaam PLUS een willekeurig getal of "seconden sinds epoch". Vraag ChatGPT gerust om hulp om dit in TypeScript voor elkaar te krijgen door je test aan te passen.
Zorg ervoor dat je deze test dus iedere keer kan draaien zonder dat je een foutmelding krijgt.

c. "Assertions" toevoegen.

Met Test assertions" voeg je controles toe aan je tests waarmee je verifieert of de test lukt of niet. Voorbeeld:

```
await expect(page.getByText('Welkom')).toBeVisible();
```

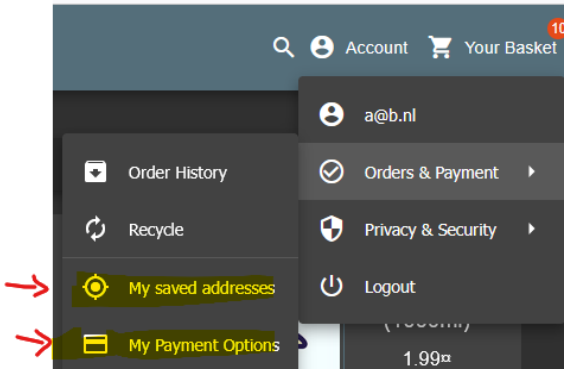
✓ Test slaagt als "Welkom" zichtbaar is.
Playwright assertions wachten automatisch tot de conditie waar is (standaard ±5 sec).



Zorg voor een **assertion**: Zorg ervoor dat je zeker weet dat je bent ingelogd. Check hiervoor bijvoorbeeld of je email adres klopt bij loginnaam.

d. Jouw gegevens nog verder uitbreiden

Zorg ervoor dat je minimaal 1 **adres** en 1 **creditcard** (geen echt nummer invullen natuurlijk) hebt toegevoegd aan je account.



e. Test2: Bestelling

Maak nu een bestelling en kijk hoe ver je kan komen om dit te automatiseren!
Bestel minimaal **3 artikelen**. En zorg ervoor dat je ook kan "uitchecken" en betalen.
Maakt daar ook een tweede assertion: Zorg ervoor dat het **bedrag** van je bestelling klopt.
Zorg dat er dan 2 seconden pauze is (Vraag ChatGPT maar)
Log daarna uit.

2.12.3 Lesstof tentamen

Wat moet je weten voor het tentamen over deze onderwerpen, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie.

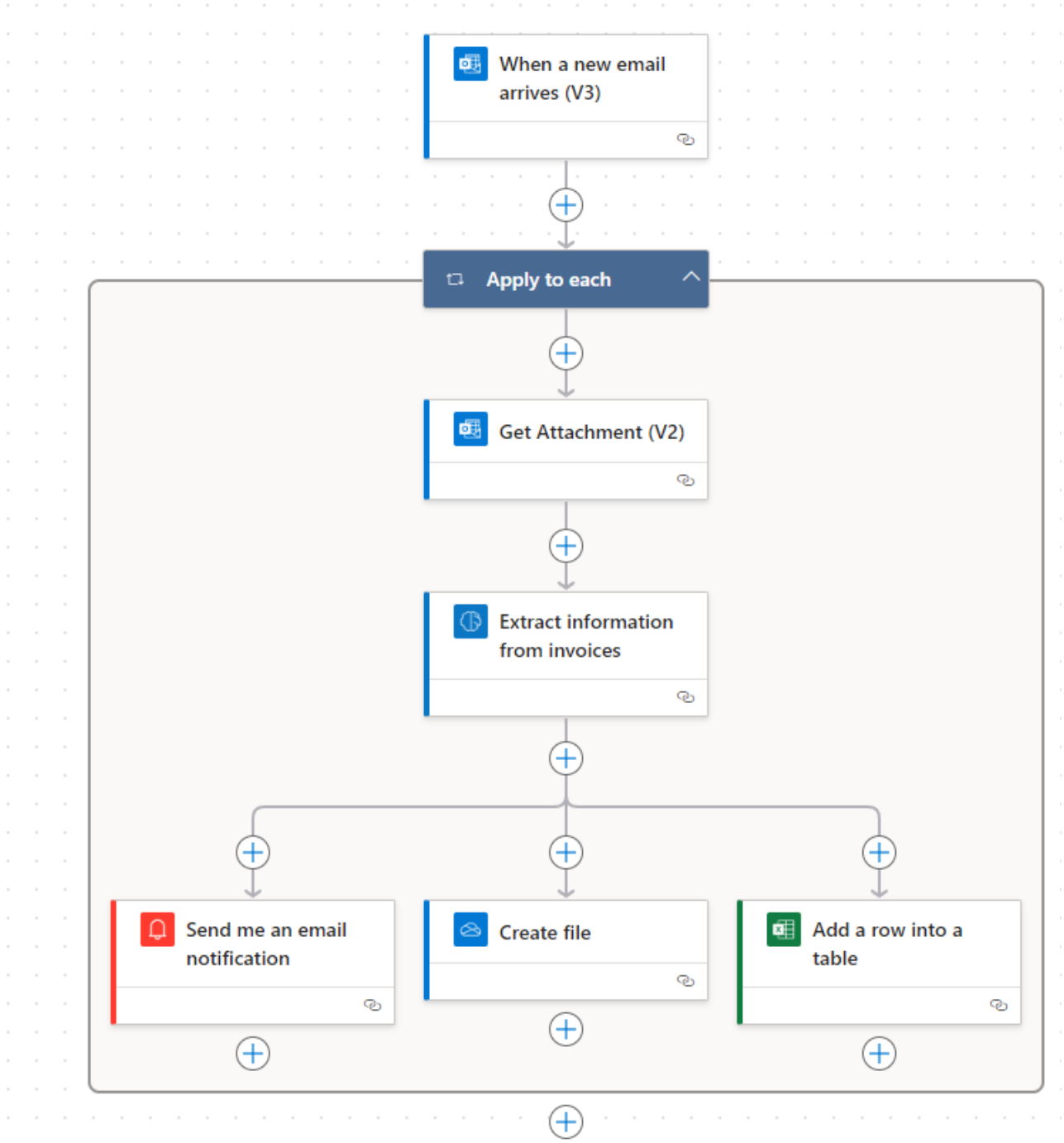
2.12.4 Informatiebronnen

- Playwright documentatie → https://playwright.dev/
- Playwright tutorial → https://playwright.dev/docs/getting-started-vscode en https://playwright.dev/docs/codegen en https://playwright.dev/docs/test-cli

2.13 Tooling 6: Power Automate

Grote organisaties in Nederland ontvangen elke dag tientallen facturen van andere organisaties. Bij een organisatie als bijvoorbeeld Intergamma (Gamma/Karwei) waren daar ruim honderd mensen dagelijks bezig om de facturen te betalen en alles te controleren. Je snapt dat het best nuttig kan zijn om dit proces te automatiseren, maar dat is nog niet zo makkelijk: elke leverancier stuurt facturen met een andere opmaak en hoe kan je dat nu automatisch lezen? Gelukkig kan AI en OCR je daarbij helpen.

Wat gaan we maken? Zodra je een e-mail met een **pdf attachment** binnenkrijgt op je HU-email adres wordt dat gezien als de "**trigger**" en beginnen de **acties** te lopen als aan de juiste **condities** is voldaan. We gaan het onderstaande bedrijfsproces "invoice processing" in stapjes opbouwen. Daar gaan we!



2.13.1 Lesopdracht

Zorg eerst dat je beschikt over de juiste randvoorwaarden, dat is:

- Kan je bij de **cloudomgeving**: https://make.powerautomate.com/?
- Heb je **OneDrive van de HU** en kan je daar met de verkenner ook bestanden in plaatsen?
- Zorg dat je de beschikking hebt over minstens **5 facturen**. Je kan ze zelf maken: https://create.microsoft.com/en-us/templates/invoices , je kan een kant en klare set downloaden van https://s3.eu-central-1.amazonaws.com/www.kwintexcloud.nl/downloads/contoso-invoices-v1.zip (of vanuit Canvas: Bestanden->Resources->consoso-invoices-v1.zip) of je zorgt zelf voor 5 facturen van dezelfde leverancier die "thuis" hebt liggen, da's wel 't leukst natuurlijk.

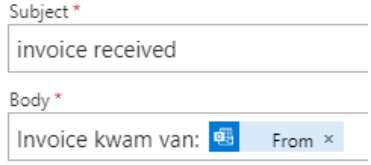
Stap1: Krijg een e-mail zodra er een factuur in je mailbox is gekomen.

Niet echt nuttig natuurlijk, "een mail krijgen zodra je een mail krijgt", maar dan weet je wel of de "trigger" werkt.

1. Ga naar https://make.powerautomate.com/ en maak een nieuwe flow. Kies voor "Automated Cloud flow"
2. Geef de flow een naam en zorg voor de juist Trigger. De trigger is natuurlijk het ontvangen van een mail. Let op, geen gmail, maar Office 365 Outlook.
3. Klik op dit blokje en zorg ervoor dat de trigger alleen afgaat als er een attachment is bijgevoegd en als het onderwerp de tekst "invoice-hu" bevat. Beetje gek misschien, maar hierdoor weet je zeker dat je normale e-mails van de HU met rust worden gelaten!
4. Zorg ook dat "Include Attachments" of yes staat.
5. Klik op Code View, zie je deze json en herken je alles?

```
{
  "type": "OpenApiConnectionNotification",
  "inputs": {
    "parameters": {
      "includeAttachments": true,
      "subjectFilter": "invoice-hu",
      "fetchOnlyWithAttachment": true
    },
    "host": {
      "apiId": "/providers/Microsoft.PowerApps/apis/shared_office365",
      "connection": "shared_office365",
      "operationId": "OnNewEmailV3"
    }
  },
  "splitOn": "@triggerOutputs()?['body/value']"
}
```

6. Dat was de "trigger", nu de "actie". Zorg ervoor dat je een "email notification" (bv office 365 send an email(v2)) ontvangt zodra de mail is ontvangen die voldoet aan de condities (namelijk een attachment en het juiste onderwerp).
7. Zorg ervoor dat het onderwerp van de notificatie mail "Invoice Received" is en dat de body de tekst krijgt: "Invoice kwam van: …" . Op de puntjes moet de afzender komen, dat doe je door een "/" in te typen en dan te klikken op "Insert Dynamic Content". De rest snap je wel.



8. Opslaan, "flow checken" en Testen maar! Wel even geduld, kan een minuutje duren.

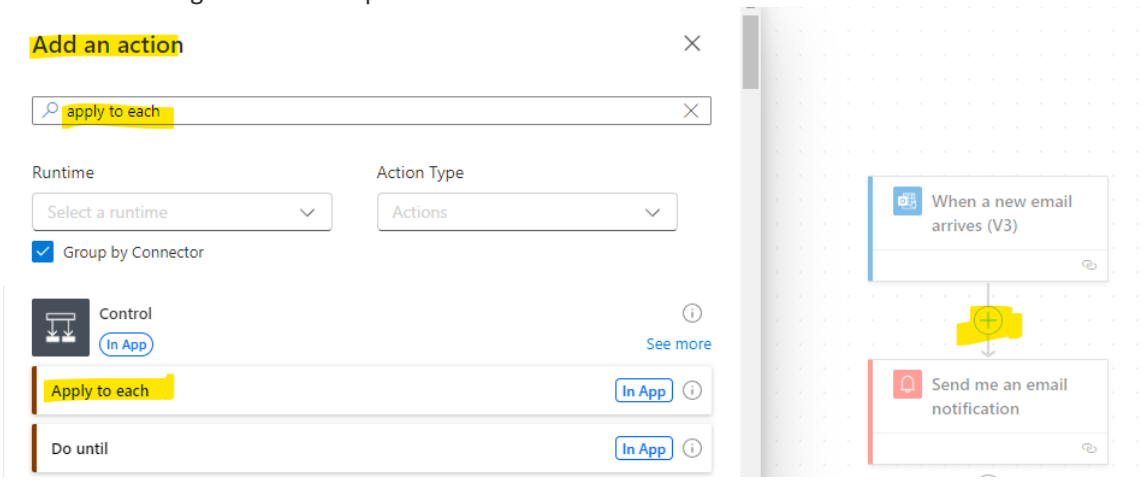


2.13.2 Weekopdracht

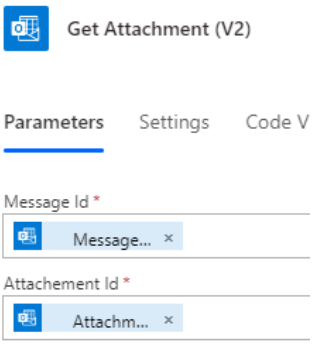
Mooi, je snapt de basis: trigger, action, condition. Nu gaan we het uitbreiden, stap voor stap. Kom je er niet uit, vraag het gerust aan Copilot. Pas op, want de Copilot kan zomaar je flow aanpassen, maar gelukkig kan je dit ook weer ongedaan maken.

Stap 2: Notificatie is mooi, maar zet de factuur ook op Onedrive.

1. Oei dit wordt lastiger. We moeten door de attachments lopen. Dat kan door een actie “apply to each” toe te voegen. Hier een tip:



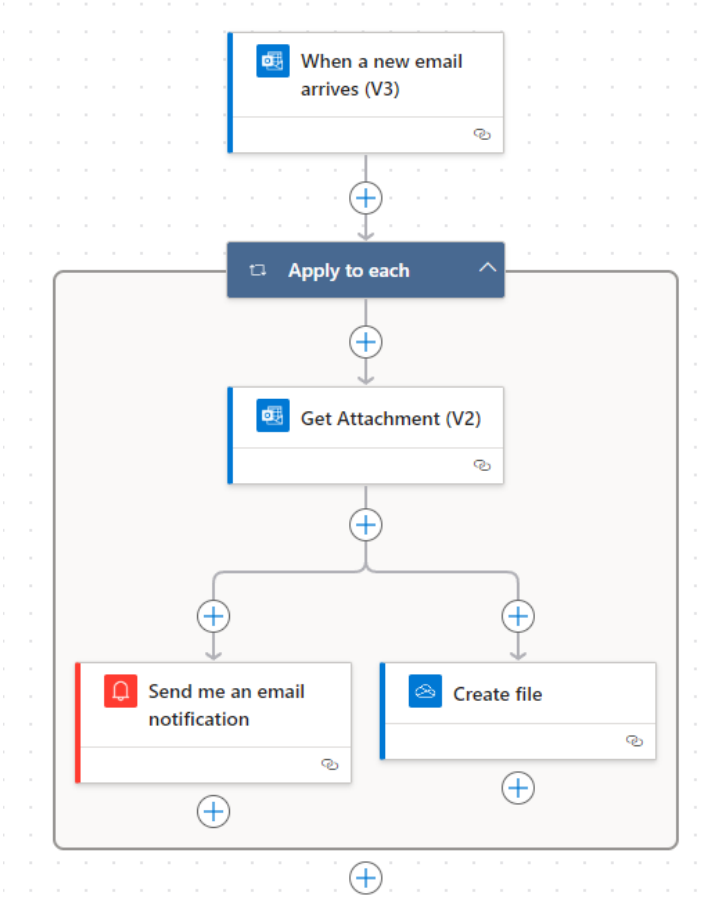
2. Voeg nu een actie “Get Attachment” toe zodat de pdf uit de mail wordt gehaald. Tips:
   - Message Id is -> “Message Id” uit de stap “When a new email arrives”, vergeet niet eerst de “/” te typen zodat je dynamische content kan toevoegen.
   - Attachment Id is -> “Attachments Attachment Id” uit de stap “When a new email arrives”, vergeet niet eerst de “/” te typen.



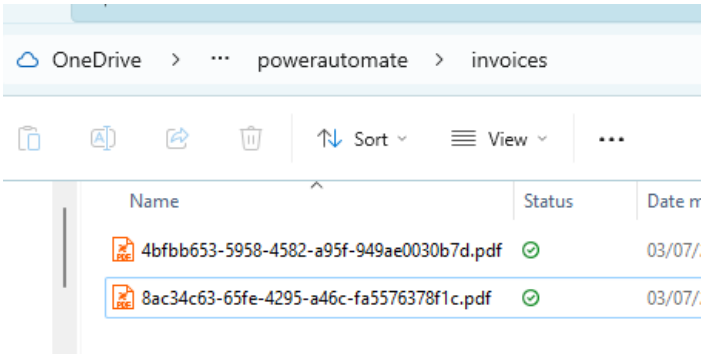
   - Dit dus:
3. Nu kan je de file opslaan met de stap “Create file” uit “OneDrive for Business”. Tips:

- Maak een map in je OneDrive for Business van de HU bv “powerautomate-invoices” en selecteer deze
- Kies als filename de guid() functie met extensie .pdf
- Kies als File Content “Content Bytes” uit de stap “Get Attachments”, vergeet niet eerst de “/” te typen.

4. Als je nu dit ongeveer hebt staan kan je het gaan testen.

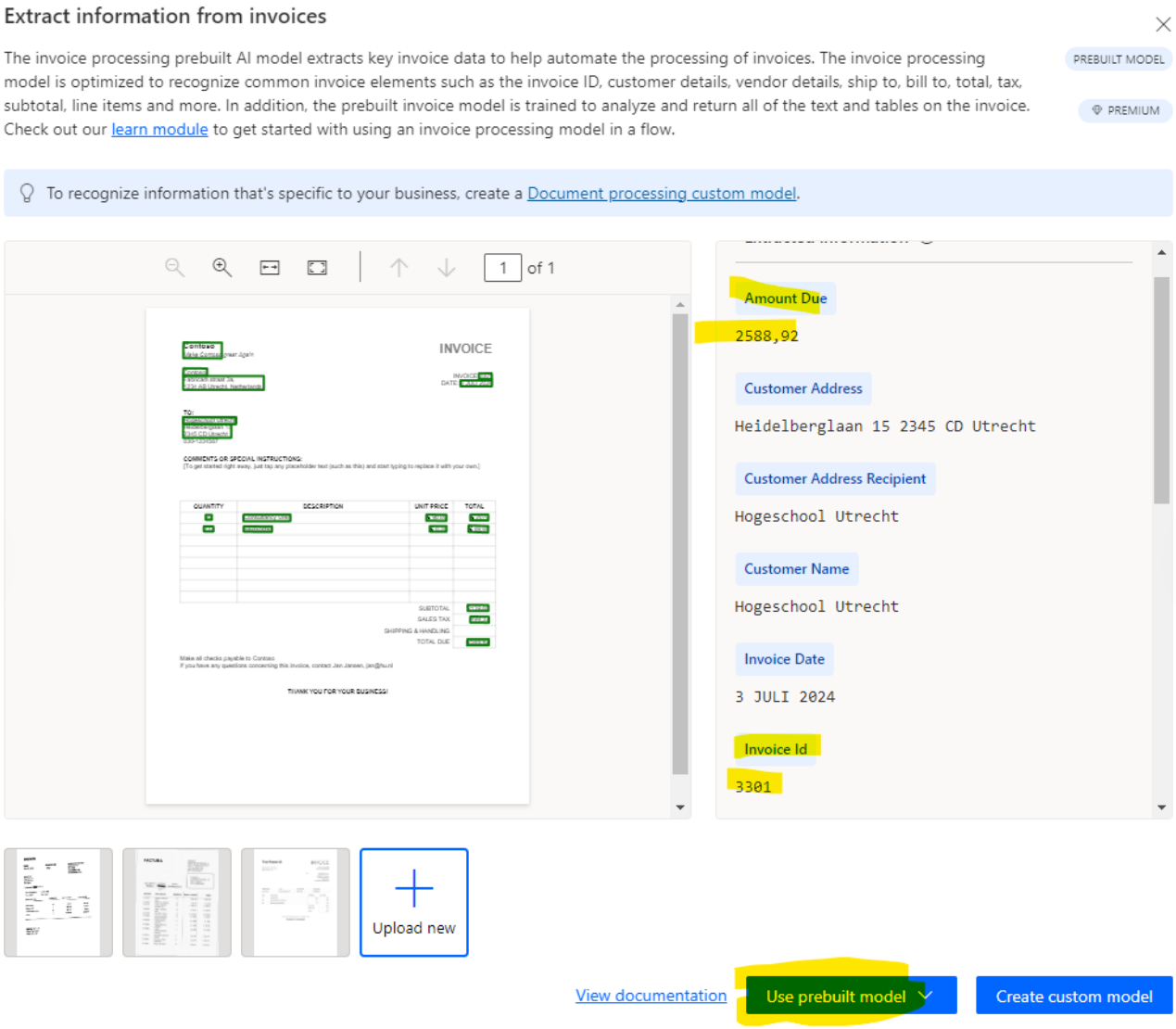


Komen de PDF’s in je map terecht?



Stap 3: Gaaf nu kijken of PowerAutomate de factuur ook kan lezen, dat doen we met een AI model.

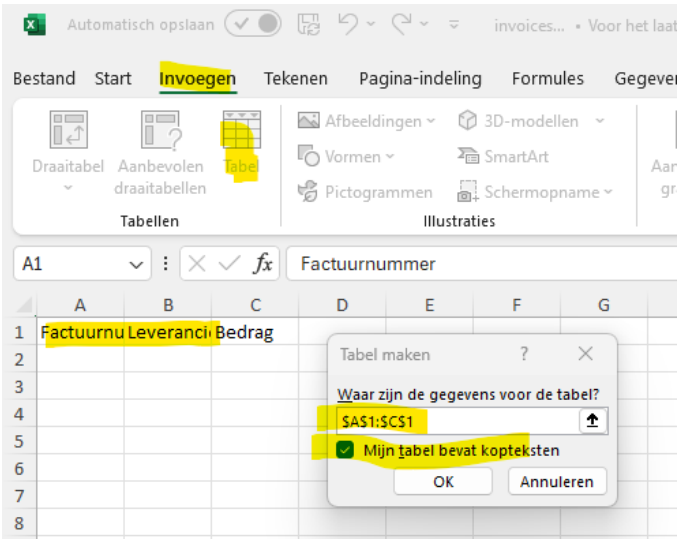
1. Klik in het hoofdmenu op “AI Hub” -> “AI Models” -> “Extract information from invoices”
2. Klik nu op “Upload new” en upload één van je facturen. Wordt de informatie goed herkend? Let vooral op “Vendor Name”, “Amount Due” en “Invoice Id”. Klopt de informatie? Zo ja, klik op “Use prebuilt model” om verder te gaan -> use in a flow.



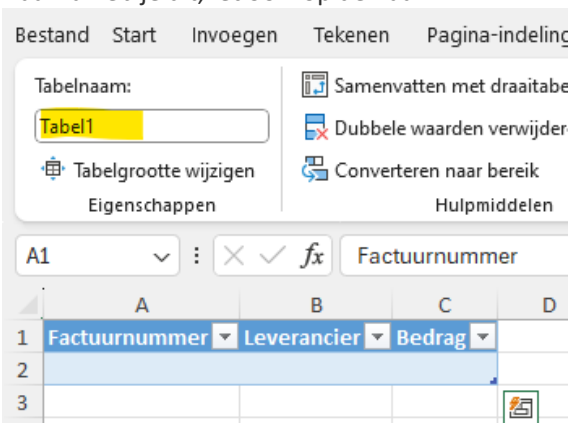
3. Ga terug naar je flow. Voeg nu na “Get Attachment” een extra stap toe, namelijk “AI Builder” en kies voor de actie “Extract infromation from invoices”. Tips:
    - Kies als Invoice File “Content Bytes” uit de stap “Get Attachments”, vergeet niet eerst de “/” te typen.

Stap 4: Mooi, de informatie uit de factuur is bekend, maar wordt niet opgeslagen. Dat gaan we nu in een Excel file plaatsen.

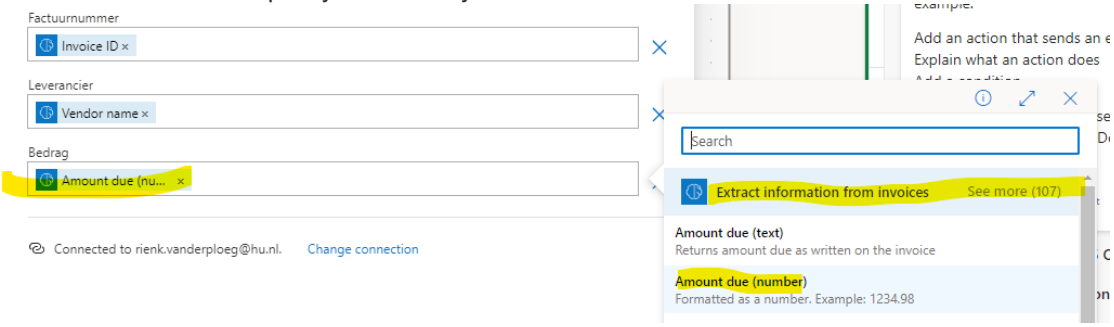
1. Maak eerst een nieuwe excel file in de root van je Onedrive van de HU. Daarin maak je een tabel. Dat doe je door eerst de kopjes “factuurnummer”, “leverancier”, “bedrag” te maken en daarna ga je naar “invoegen” -> “tabel”, selecteer de koptekksten en maak de tabel.



Daarna heb je dit, let ook op de naam:

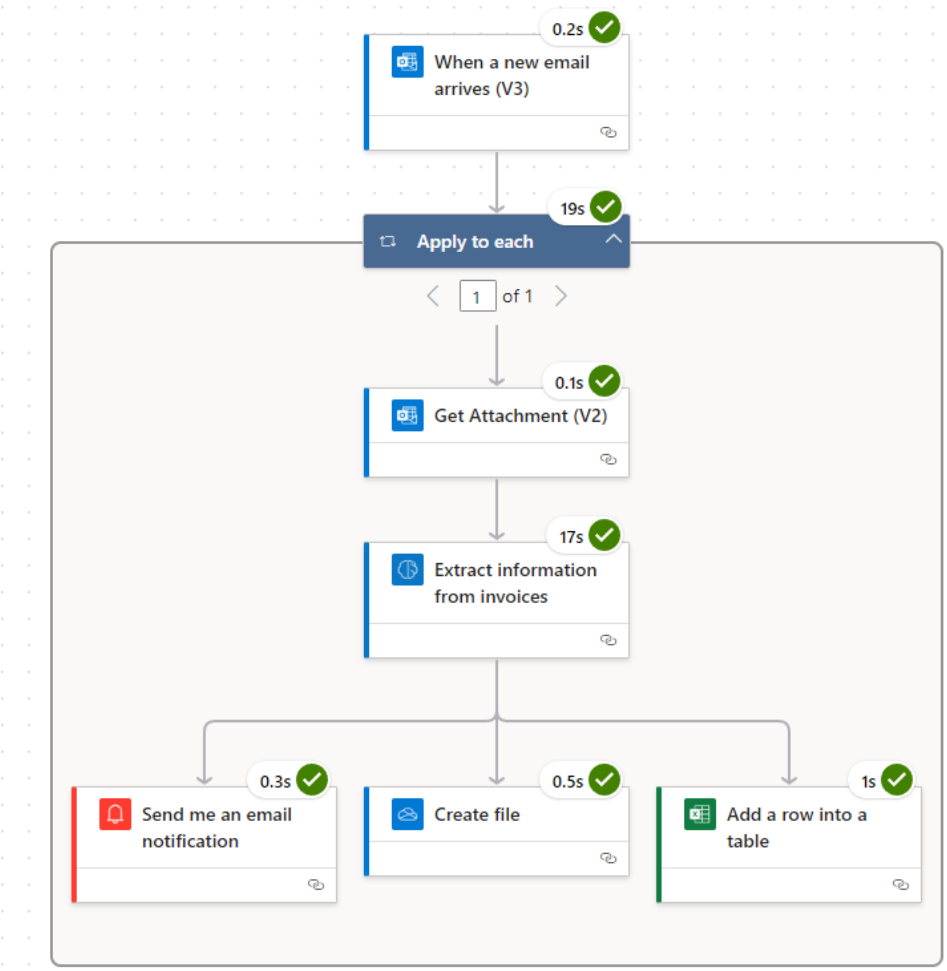


2. Nu willen we deze excel graag vullen met informatie uit de invoice die AI voor je heeft gedetecteerd. Voeg een parallelle actie toe met de naam: “Add a row into a table” van Excel Online. Tips:
   - Location: OneDrive for Business
   - Document Library: “Documenten”
   - File: Selecteer je Excel file die je net hebt gemaakt.
   - Table: “tabel1”, dus de naam van de tabel die je hebt gemaakt.
   - Advanced parameters: “Show all” als het goed is zie je nu al je kolommen staan. “Map” nu de kolom in excel op de juist data in je invoice.



3. Test de flow, kan ook met een mail met meerdere facturen als attachment. Heb je het onderstaande gebouwd en komt de informatie in de excel file? (Note: bij factuur 2, 4 en 5 wordt het bedrag niet gevonden en ook de leveranciernaam is niet altijd ok. Dit is wel oplosbaar, maar dat gaat te ver voor deze weekopdracht).

	A	B	C
1	Factuurnummer	Leverancier	Bedrag
2	3301	Contoso Make Contoso	2588,92
3	3301	Contoso Make Contoso	2588,92
4	3304	Contoso Make Contoso great Again	
5	3305	Contoso Make Contoso great Again	
6	3301	Contoso Make Contoso	2588,92
7	3302	Contoso Make Contoso great Again	
8	3303	Contoso Make Contoso great Again	1125,30



Stap 5: Voeg nu zelf nog een stap toe naar keuze.

Je weet nu redelijk goed wat er mogelijk is met Power Automate. Toch ben je stap-voor-stap door het proces heen geholpen. Kan je de flow nu zelf uitbreiden met iets van jezelf?

Dat kan vanalles zijn:

- de data uit de invoice in een tekst file zetten,
- versturen per mail,
- in een database zetten,
- naar Twitter sturen
- Berichtje in teams zetten.

Kies zelf iets….

2.13.3 Lesstof tentamen

Wat moet je weten voor het tentamen over dit onderwerp, dus hoe kan je je het beste voorbereiden?

- **Leer** alle informatie die je vindt in de powerpoint presentatie.
- **Zorg dat je het lab hebt gedaan** zodat je tijdens het tentamen daar vragen over kan beantwoorden.

2.13.4 Informatiebronnen

- Powerautomate site: https://make.powerautomate.com/
- Officiele documentatie: https://learn.microsoft.com/en-us/power-automate/getting-started
- Lab is gebaseerd op: https://nanonets.com/blog/microsoft-power-automate-tutorial-guide/

2.14 Bijlage1: Antwoorden 2.1.1, opdracht 1

```
"""
Herhalingsopdrachten PYTHON
1) Zet 5 namen in een lijst en ga er met een for-loop doorheen.
    Print alle namen onder elkaar.
"""
l=['jan','piet','ali','amir','truus']
for i in l:
    print(i)

"""
2) Vraag een gebruiker om een getal.
    Als het getal > 100 dan print je "GROOT", anders print je "klein" op het scherm.
"""
# g=int(input("Geef een getal: "))
# if g>100:
#     print("GROOT")
# else:
#     print("klein")

"""
3) Een auto is rood, de trein is geel en de ambulance ook trouwens.
    Zet deze informatie in een dictionary en print deze vervolgens op het scherm.
"""
d={'auto':'rood', 'trein':'geel', 'ambulance':'geel'}
print(d)
for i in d:
    print(i,"-->",d[i])

"""
4) Gegeven de string: a="The client connects to the server by calling socket()".
    Print karakter 5 t/m 10 op het scherm.
"""
a="The client connects to the server by calling socket()"
print(a[4:10])

"""
5) Gegeven de string uit de vorige opdracht:
    print vanaf de laatste 8 karakters tot het einde.
"""
a="The client connects to the server by calling socket()"
print(a[-8:])

"""
6) Print alle letters uit de vorige opdracht met een spatie tussen elke letter
    (tip: gebruik for-loop en bij de printopdracht: end=" ")
"""
a="The client connects to the server by calling socket()"
for i in a:
    print(i, end=" ")

"""
7) Maak een tuple van 10 getallen en print deze gesorteerd van klein naar groot op het scherm.
Print daaronder het totaal van alle getallen.
"""
t=(7,55,7,0,22,3,3,4456,8,6)
print(sorted(t))
print(sum(t))

"""
8) Maak een eindeloze loop.
    Vraag daarin de gebruiker om input.
    Als de gebruiker "stop" invoert, dan moet je uit de loop springen en is het programma afgelopen.
"""
while(True):
    s=input("type iets...")
    if s == "stop":
        break

"""
9) Maak een functie.
    De functie krijgt een getal mee en
    geeft het dubbele van dit getal terug als het wordt aangeroepen.
    Roep de functie aan voor de getallen 3, 60 en -3 en print het resultaat op het scherm.
"""
```

```
def f(x):
    return x+x

print(f(3))
print(f(60))
print(f(-3))

"""
10) Vraag de gebruiker om een getal.
    Probeer het getal 100 daarna door de opgegeven invoer te delen en print het resultaat.
    Als de invoer "0" was, dan crashed je programma. Zorg dat dit niet gebeurt (gebruik excepties).
"""
g=input("Geef de noemer ")
try:
    print(100/int(g))
except Exception as e:
    print("Delen door 0 mag niet:", e)

"""
11) Maak met notepad een tekstbestand van 4 regels. Open dit bestand in Python en print regel voor
regel op het scherm. Zorg ervoor dat er een regelnummer staat voor elke uitvoerregel. Dus bv zo:
1 blabladfalf dsfjsflksjf
2 ddddf dsfjsklfjklsf
3 sfdkfjdkfjdkfjdkfdkfjdkf
"""
import os
print(os.getcwd())
t=1
f=open('Week1\\README.txt')
b=f.readlines()
t=1
for i in b:
    print(t,i, end='')
    t+=1
f.close()

"""
12)  Maak werkende.
"""
def doe_iets(x):
    if x < 10:
        y = "x is kleiner dan 10"
    elif x>=10 and x<20:
        y = "x tussen 10 en 20"
    else:
        y = "x is groter dan 19"
    return y
getal=int(input("Geef een getal "))
z=doe_iets(getal)
print(z)

"""
13)    Maak een lijst met daarin twee lijstjes met ieder 3 getallen.
    print uit de tweede lijst het tweede getal.
"""
p=[[1,2,3,4],[11,22,33,44]]
print(p[1][1])

"""
14)    Print de huidige werkdirectory (linux: pwd). Tip gebruik de module os en functie getcwd…
"""
import os
print("De huidige directory is:",os.getcwd())
```

2.15 Bijlage2: Antwoorden 2.2.1, functies

```
# https://www.w3resource.com/python-exercises/

### 1 ### easy
l=(8, 2, 3, -1, 7)
def find_highest(my_list):
    x=max(my_list)
    return x
print(find_highest(l))


### 2 ### easy
def calculate_birth_year(age):
    year_now=2022
    return year_now-age
print(calculate_birth_year(51))


### 3 ### easy
import datetime
def calculate_birth_year(age):
    year_now = datetime.datetime.today().year
    return year_now-age
print(calculate_birth_year(51))


### 4 ### easy
def check_number(number,start,stop):
    if number > start and number < stop:
        return 'yes'
    else:
        return 'no'
r=check_number(23,10,100)
print("Is number in given range?:", r)


### 5 ### easy
def is_even_num(l):
    even_number = []
    for n in l:
        if n % 2 == 0:
            even_number.append(n)
    return even_number
l=[1, 2, 3, 4, 5, 6, 7, 8, 9]
print(is_even_num(l))
```

```
### 6 ### difficult
l=(8, 2, 3, -1, 7)
def multiply_all(my_list):
    total=1
    for i in my_list:
        total *=i
    return total
print(multiply_all(l))

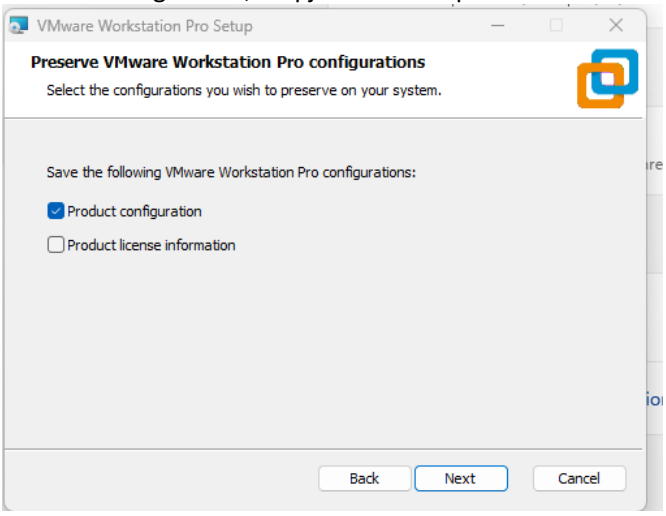

### 7 ### medium/difficult
s="1234abcd"
l=len(s)
for i in range(l):
    print(s[l-i-1],end='')
print()


### 8 ### difficult
def check_prime(number):
    flag=False
    for i in range(number):
        if i == 1 or i ==0:
            continue
        elif number % i == 0:
            flag=True
            print(number,' is deelbaar door:', i)
    return flag

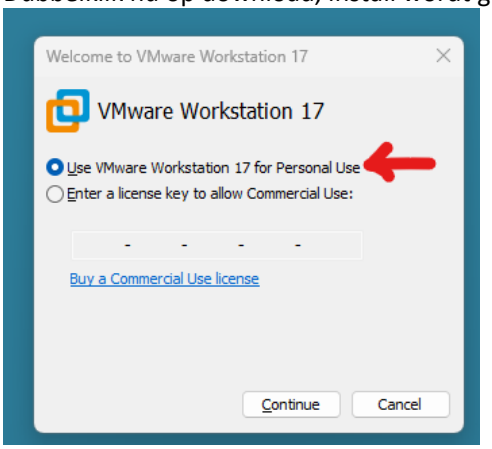
g=7  # 70=geen priemgetal | 7 is een priemgetal
r=check_prime(g)
if r==False:
    print(g," = een priemgetal")
else:
    print(g," = geen priemgetal")
```

2.16 Bijlage3: Install VMware Workstation

- Register bij support.broadcom.com maar niet met hu-email-adres zoals bekend bij Broadcom/vmware (bij sommigen werkt dit, anderen niet. Gebruik dus evt. een privé e-mail adres.
- Na registratie, ga naar: https://support.broadcom.com/group/ecx/productdownloads?subfamily=VMware%20Workstation%20Pro&freeDownloads=true
- Kies latest and greatest (personal use windows, 17.5.2 of later)
- agree to terms and conditions
- klik op download, dat lukt niet.
- vul extra persoonlijke data in met wat je kwijt wil en submit.
- klik opnieuw op download. Lukt nu wel.
- Deinstalleer bestaande VMware installatie, maar behoud productconfiguratie (om niet al je netwerkconfiguraties, mapjes en vm's opnieuw toe te hoeven voegen!).



- Dubbelklik nu op download, install wordt gestart. Kies bij starten voor:



- Klaar!